Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

Chapter 4: Transform-and-conquer

Algorithms Analysis and Design (CO3031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn

Semester 2 – 2022-2023

Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.

2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies*.

3. Able to discuss NP-completeness.

References

[1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.

[2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, *3rd Edition*, Pearson, 2012.

[3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.

[4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, TheBenjamin/Cummings Publishing, 1993.

Course Outline

- Chapter 1. Fundamentals
- Chapter 2. Divide-and-Conquer Strategy
- Chapter 3. Decrease-and-Conquer Strategy
- **Chapter 4. Transform-and-Conquer Strategy**
- Chapter 5. Dynamic Programming and Greedy Strategies
- Chapter 6. Backtracking and Branch-and-Bound
- Chapter 7. NP-completeness
- Chapter 8. Approximation Algorithms

Chapter 4. Transform-and-Conquer Strategy

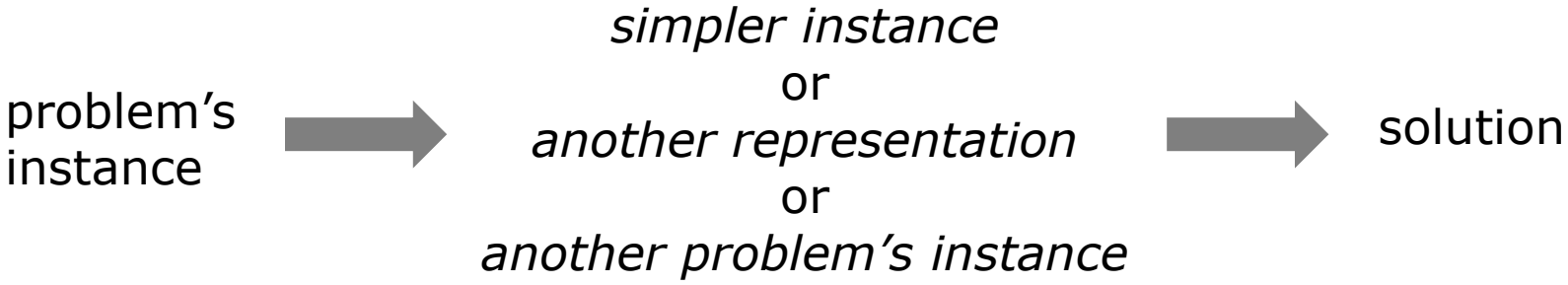
- 4.1. Transform-and-conquer strategy
- 4.2. Gaussian Elimination for solving a system of linear equations
- 4.3. Heaps and heapsort
- 4.4. Horner's rule for polynomial evaluation
- 4.5. String matching by Rabin-Karp algorithm

4.1. Transform-and-conquer strategy

- *Transform-and-conquer* works in a two-stage procedure.
  - *Transformation*: the problem's instance is modified to be more amenable to solution.
  - *Conquer*: the modified instance is solved.
- Three major variations:
  - ***Instance simplification***: transformation to a simpler or more convenient instance of the same problem
  - ***Representation change***: transformation to a different representation of the same instance
  - ***Problem reduction***: transformation to an instance of a different problem for which an algorithm is already available

4.1. Transform-and-conquer strategy

- Three major variations:
  - ***Instance simplification***: Gaussian elimination to solve a system of linear equations, AVL tree
  - ***Representation change***: heapsort, Horner's rule to evaluate polynomials, Rabin-Karp for string matching, B-tree
  - ***Problem reduction***: an algorithm for computing the least common multiple by using Euclid's algorithm for finding the greatest common divisor, graph coloring for edges using the algorithm for vertices



Transform-and-conquer

Figure 6.1, [2], pp. 197.

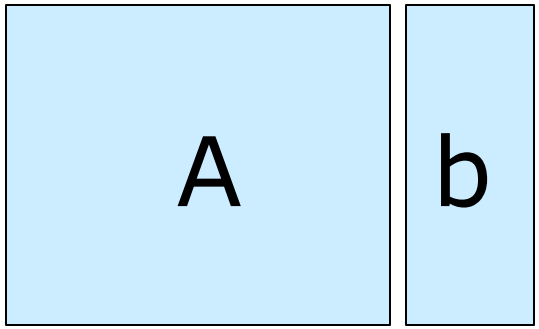
4.2. Gaussian Elimination for solving a system of linear equations

- Gaussian Elimination algorithm helps us solve a system of *n* linear equations in *n* unknowns.
  - *transform* a system of *n* linear equations in *n* unknowns to an equivalent system (i.e., a system with the same solution as the original one) with an *upper-triangular coefficient matrix*, a matrix with all zeros below its main diagonal
  - → Instance simplification of the problem
  - → The system with an upper-triangular coefficient matrix is then easily solved by *backward substitutions*.

4.2. Gaussian Elimination for solving a system of linear equations

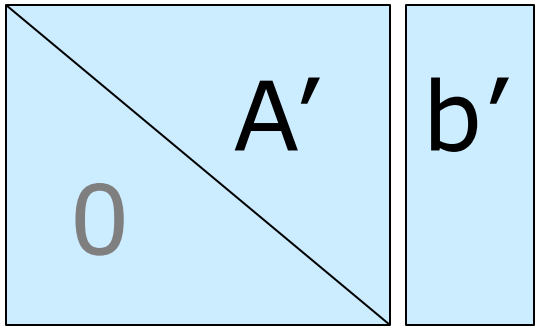
- Given a system of *n* linear equations in *n* unknowns

$$a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1$$
$$a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2$$
$$\vdots$$
$$a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n$$



Gaussian Elimination

$$a'_{11}x_1 + a'_{12}x_2 + \ldots + a'_{1n}x_n = b'_1$$
$$a'_{22}x_2 + \ldots + a'_{2n}x_n = b'_2$$
$$\vdots$$
$$a'_{nn}x_n = b'_n$$



4.2. Gaussian Elimination for solving a system of linear equations

GaussElimination(A[1..n,1..n],b[1..n])

```
for i := 1 to n do A[i,n+1] := b[i];
for i := 1 to n -1 do                    //row i
   for j := i + 1 to n do                //row i+1..n
      for k:= i to n+1 do                //column i..n+1
        A[j,k] := A[j,k]-A[i,k]*A[j,i]/A[i,i];
```

Note: *partial pivoting* to deal with *small* |A[i,i]|

4.2. Gaussian Elimination for solving a system of linear equations

- Solve the following system of 3 linear equations

$$2x_1 - x_2 + x_3 = 1$$
$$4x_1 + x_2 - x_3 = 5$$
$$x_1 + x_2 + x_3 = 0$$

2	-1	1	1	
4	1	-1	5	row 2 – (4/2) row 1
1	1	1	0	row 3 – (1/2) row 1

2	-1	1	1	
0	3	-3	3	row 3 – (1/2) row 2
0	3/2	1/2	-1/2	

2	-1	1	1
0	3	-3	3
0	0	2	-2

⇒ *Backward substitution*:

$x_3 = (-2)/2 = -1$

$x_2 = (3-(-3).x_3)/3 = 0$

$x_1 = (1-(1.x_3+(-1).x_2))/2 = 1$

4.2. Gaussian Elimination for solving a system of linear equations

- Solve the following system of 3 linear equations

$$2x_1 - x_2 + x_3 = 1$$
$$4x_1 + x_2 - x_3 = 5$$
$$x_1 + x_2 + x_3 = 0$$

2	-1	1	1
4	1	-1	5
1	1	1	0

row *j*, A[*j*][*i*], A[*i*][*i*], row *i*

row 2 – (4/2) row 1
row 3 – (1/2) row 1

2	-1	1	1
0	3	-3	3
0	3/2	1/2	-1/2

row 3 – (1/2) row 2

2	-1	1	1
0	3	-3	3
0	0	2	-2

⇒ *Backward substitution*:

$x_3 = (-2)/2 = -1$
$x_2 = (3-(-3).x_3)/3 = 0$
$x_1 = (1-(1.x_3+(-1).x_2))/2 = 1$

4.2. Gaussian Elimination for solving a system of linear equations

```
BetterGaussElimination(A[1..n,1..n],b[1..n])
for i := 1 to n do A[i,n+1] := b[i];
for i := 1 to n -1 do
   pivotrow := i;
   for j := i+1 to n do
      if |A[j,i]| > |A[pivotrow,i]| then pivotrow:= j;
   for k:= i to n+1 do
      swap(A[i,k], A[pivotrow,k]);
   for j := i + 1 to n do
      temp:= A[j,i]/A[i,i];
      for k:= i to n+1 do
        A[j,k] := A[j,k]-A[i,k]*temp;
```

4.2. Gaussian Elimination for solving a system of linear equations

- Solve the following system of 3 linear equations

$$\begin{aligned} x_2 + x_3 &= -1 \\ 4x_1 + x_2 - x_3 &= 5 \\ 2x_1 - x_2 + x_3 &= 1 \end{aligned}$$

pivoting =>

$$\begin{aligned} 4x_1 + x_2 - x_3 &= 5 \\ x_2 + x_3 &= -1 \\ 2x_1 - x_2 + x_3 &= 1 \end{aligned}$$

$$\begin{matrix} 4 & 1 & -1 & 5 \\ 0 & 1 & 1 & -1 \\ 2 & -1 & 1 & 1 \end{matrix}$$

row 2 – (0/4) row 1

row 3 – (2/4) row 1

$$\begin{matrix} 4 & 1 & -1 & 5 \\ 0 & 1 & 1 & -1 \\ 0 & -3/2 & 3/2 & -3/2 \end{matrix}$$

pivoting =>

$$\begin{matrix} 4 & 1 & -1 & 5 \\ 0 & -3/2 & 3/2 & -3/2 \\ 0 & 1 & 1 & -1 \end{matrix}$$

row 3 – (1/(-3/2)) row 2

$$\begin{matrix} 4 & 1 & -1 & 5 \\ 0 & -3/2 & 3/2 & -3/2 \\ 0 & 0 & 2 & -2 \end{matrix}$$

$\Rightarrow$ *Backward substitution*:

$x_3 = (-2)/2 = -1$

$x_2 = ((-3/2)-(3/2).x_3)/(-3/2) = 0$

$x_1 = (5-((-1).x_3+1.x_2))/4 = 1$

4.2. Gaussian Elimination for solving a system of linear equations

- Time complexity to *transform* a system of *n* linear equations in *n* unknowns to an equivalent system with an upper-triangular coefficient matrix
  - Abstract operation: *multiplication*
  - Worst, Average, Best cases

$$C_n = \frac{n(n-1)(2n+5)}{6} = O(n^3)$$

4.2. Gaussian Elimination for solving a system of linear equations

- Time complexity of Gaussian Elimination

```
GaussElimination(A[1..n,1..n],b[1..n])

for i := 1 to n do A[i,n+1] := b[i];
for i := 1 to n -1 do
   for j := i + 1 to n do
      for k:= i to n+1 do
         A[j,k] := A[j,k]-A[i,k]*A[j,i]/A[i,i];
```

For all cases:

$$C_n = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \sum_{k=i}^{n+1} 1 = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n+1-i+1) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n-i+2)$$

4.2. Gaussian Elimination for solving a system of linear equations

- Time complexity of Gaussian Elimination
  - Abstract operation: *multiplication*
  - Worst, Average, Best cases

$$C_n = \frac{n(n-1)(2n+5)}{6} = O(n^3)$$

For all cases:

$$C_n = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \sum_{k=i}^{n+1} 1 = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n+1-i+1) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n-i+2)$$

$$C_n = \sum_{i=1}^{n-1} (n-i+2)(n-(i+1)+1) = \sum_{i=1}^{n-1} (n^2+2n-2ni\ -\ 2i+i^2)$$

$$C_n = (n^2+2n)(n-1-1+1) - \frac{(2n+2)n(n-1)}{2} + \frac{(n-1)n(2n-1)}{6}$$

4.2. Gaussian Elimination for solving a system of linear equations

- Time complexity of Gaussian Elimination
  - Abstract operation: *multiplication*
  - Worst, Average, Best cases

$$C_n = \frac{n(n-1)(2n+5)}{6} = O(n^3)$$

For all cases:

$$C_n = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \sum_{k=i}^{n+1} 1 = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n+1-i+1) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n-i+2)$$

$$C_n = (n^2+2n)(n-1-1+1) - \frac{(2n+2)n(n-1)}{2} + \frac{(n-1)n(2n-1)}{6}$$

$$C_n = \frac{(n-1)(6n^2+12n-6n^2-6n+2n^2-n)}{6} = \frac{(n-1)(2n^2+5n)}{6} = O(n^3)$$

4.2. Gaussian Elimination for solving a system of linear equations

- Time complexity of Gaussian Elimination
  - Abstract operation: *multiplication*
  - Worst, Average, Best cases

$$C_n = \frac{n(n-1)(2n+5)}{6} = O(n^3)$$

For all cases:

$$C_n = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \sum_{k=i}^{n+1} 1 = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n+1-i+1) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n-i+2)$$

$$C_n = (n^2+2n)(n-1-1+1) - \frac{(2n+2)n(n-1)}{2} + \frac{(n-1)n(2n-1)}{6}$$

$$C_n = \frac{(n-1)(6n^2+12n-6n^2-6n+2n^2-n)}{6} = \frac{n(n-1)(2n+5)}{6} = O(n^3)$$

4.2. Gaussian Elimination for solving a system of linear equations

- Your turn:

  4.2.1. Solve each following system of 3 linear equations.

  4.2.2. Write an algorithm for the back-substitution stage. What is its complexity?

$x_1 + x_2 + x_3 = 2$	$2x_1 + x_2 - 7x_3 = -2$	$x_2 + 2x_3 = 5$
$2x_1 + x_2 + x_3 = 3$	$x_1 - 2x_2 + 4x_3 = 4$	$2x_1 - x_2 + 6x_3 = 1$
$x_1 - x_2 + 3x_3 = 8$	$5x_1 - 3x_2 + x_3 = 8$	$3x_1 + 2x_2 - x_3 = 2$
(S1)	(S2)	(S3)

4.3. Heaps and heapsort

- Heap: a *priority queue* where the element with the highest priority is dequeued as needed.
- Max-heap
  - *Array* representation: a[1], a[2], .., a[*n*]

    a[*i*] ≥ a[2*i*] and a[*i*] ≥ a[2*i*+1] for each *i*>0
  - *Tree* representation: a nearly complete binary tree
    - all its levels are full except possibly the last level, where only some rightmost leaves may be missing.
    - the key at each node is greater than or equal to the keys at its children.

4.3. Heaps and heapsort

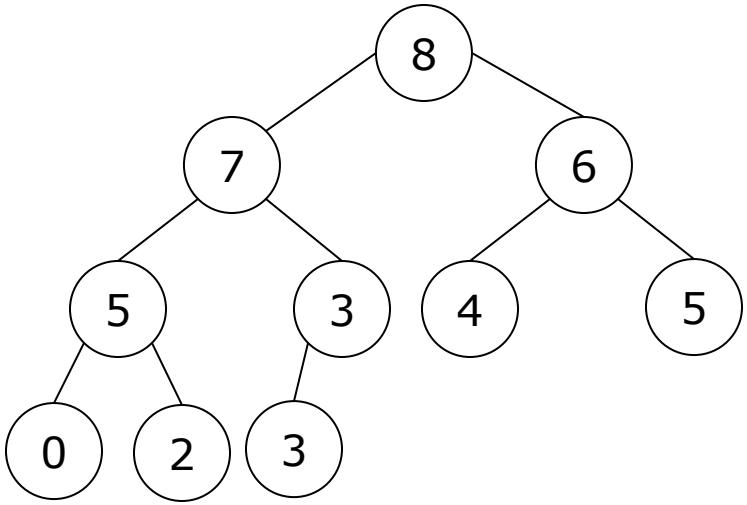
- Max-heap

Array representation

i	1	2	3	4	5	6	7	8	9	10
a	8	7	6	5	3	4	5	0	2	3

parents leaves

Tree representation



parent at *i*	→	leaves at 2*i*, 2*i* + 1
leaf at *i*	→	parent at *i* div 2

4.3. Heaps and heapsort

- Operations on a heap of *n* elements
  - *Upheap*: insertion of a new element into a heap
  - *Downheap*: remove the minimum/maximum (top-priority) element from a heap

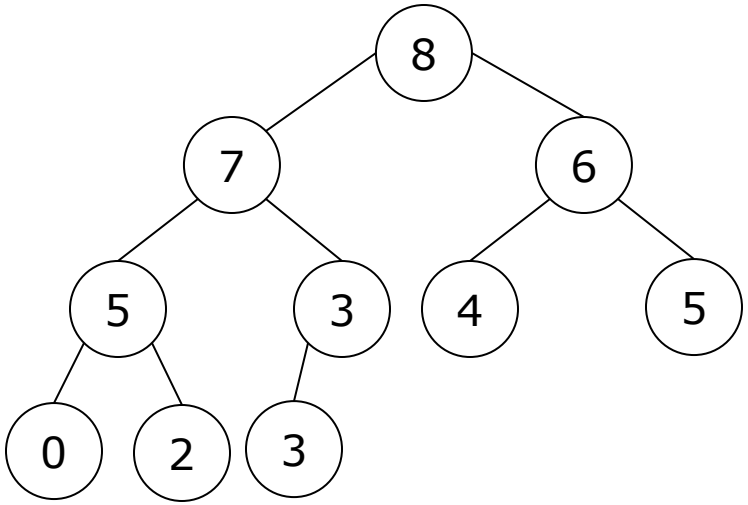
4.3. Heaps and heapsort

- Upheap on a heap of *n* elements

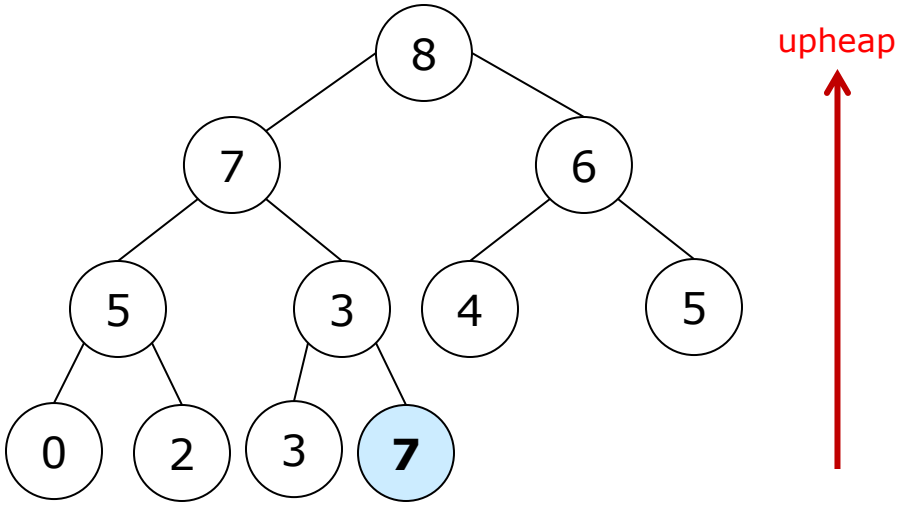
```
procedure upheap(k:integer)
var  v: integer;
begin
        v :=a[k]; a[0]:= maxint;
        while a[k div 2] <= v do
        begin a[k]:= a[k div 2 ]; k:=k div 2 end;
    a[k]:= v
end;

procedure insert(v:integer);
begin
        n:= n+1; a[n] := v ; upheap(n)
end;
```

4.3. Heaps and heapsort

- Operations on a heap of *n* elements

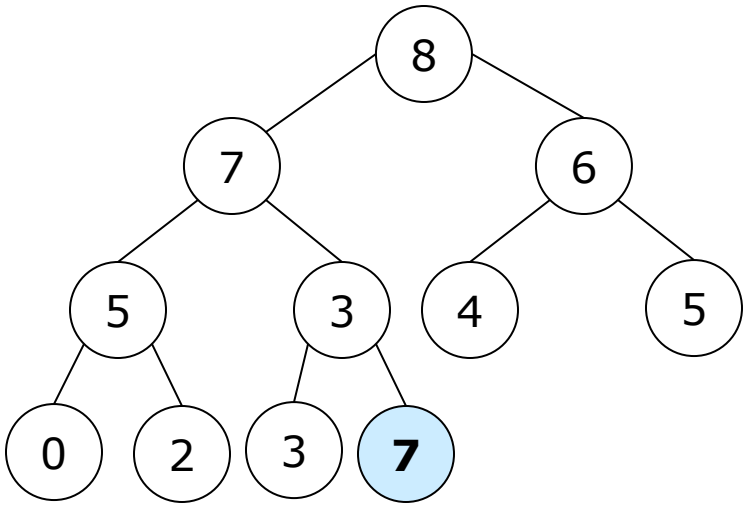


Insert 7 into this max-heap?

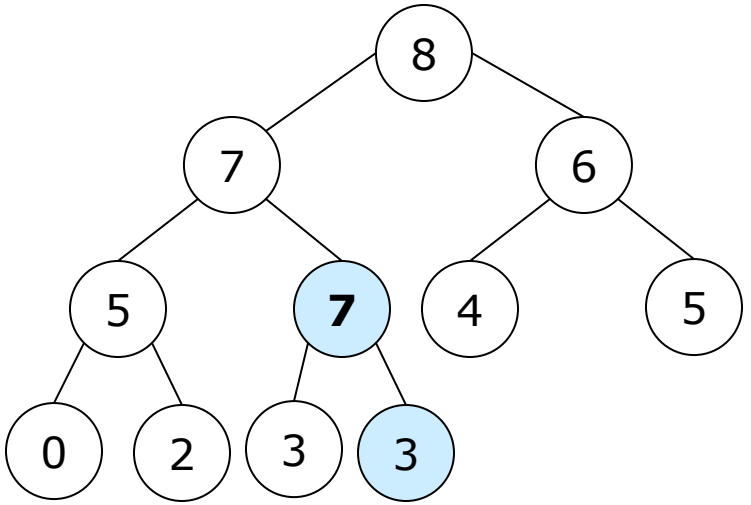


4.3. Heaps and heapsort

- Operations on a heap of *n* elements

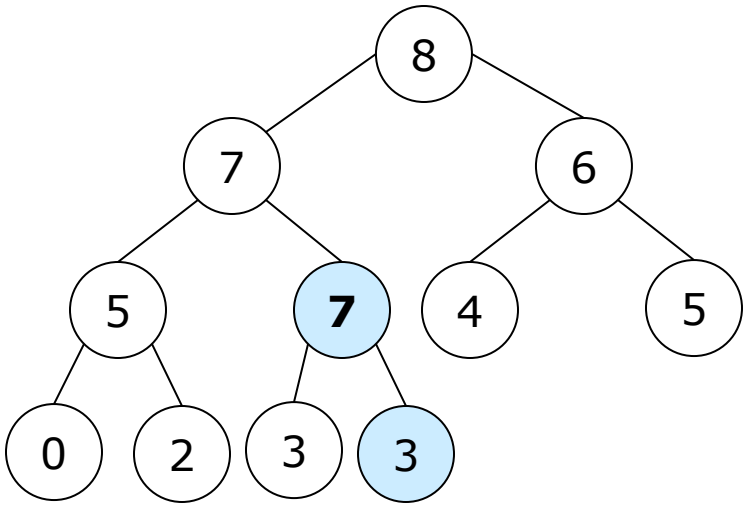


Insert 7 into this max-heap?

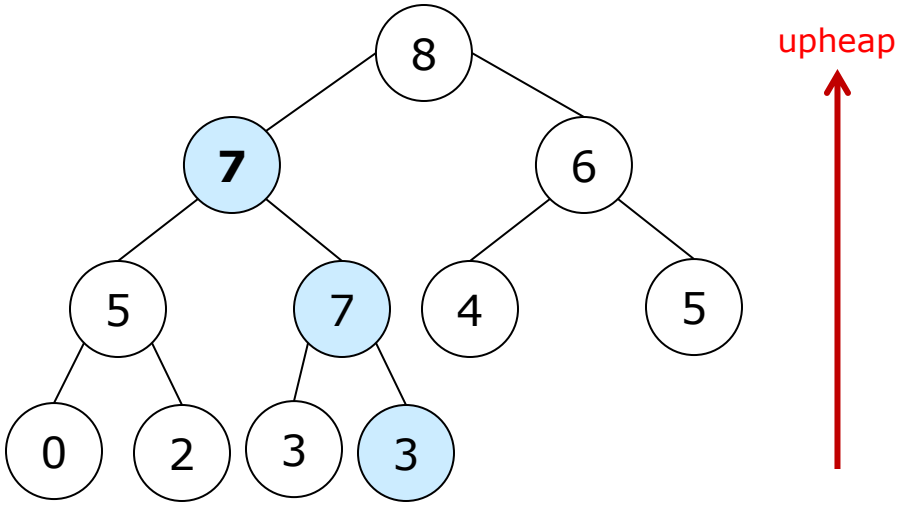


4.3. Heaps and heapsort

- Operations on a heap of *n* elements

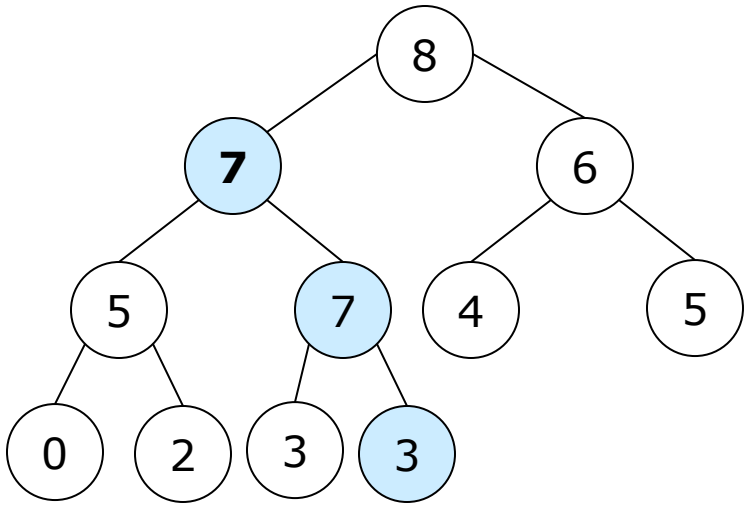


Insert 7 into this max-heap?

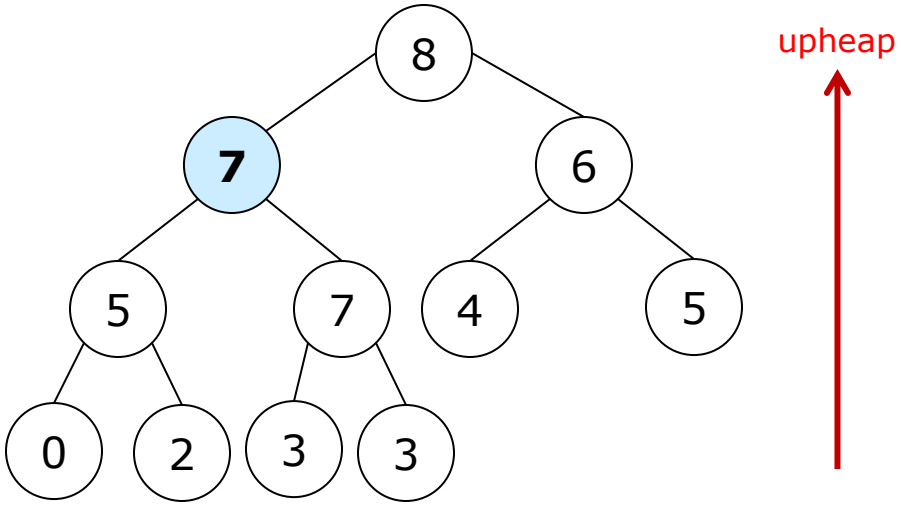


4.3. Heaps and heapsort

- Operations on a heap of *n* elements

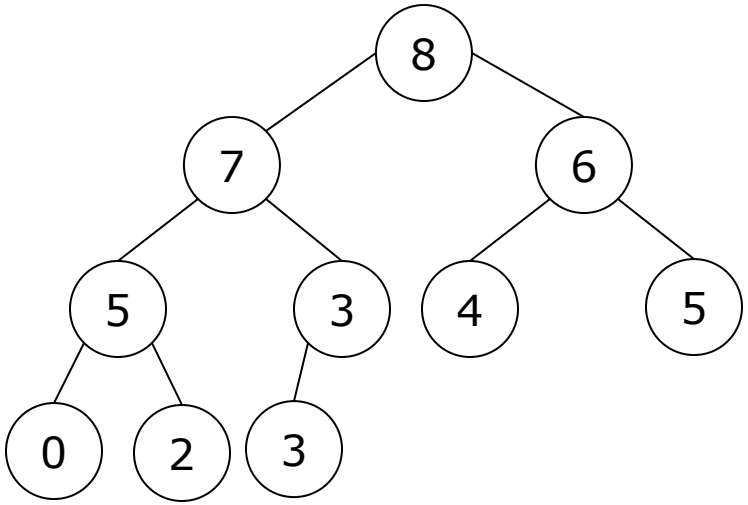


Insert 7 into this max-heap?

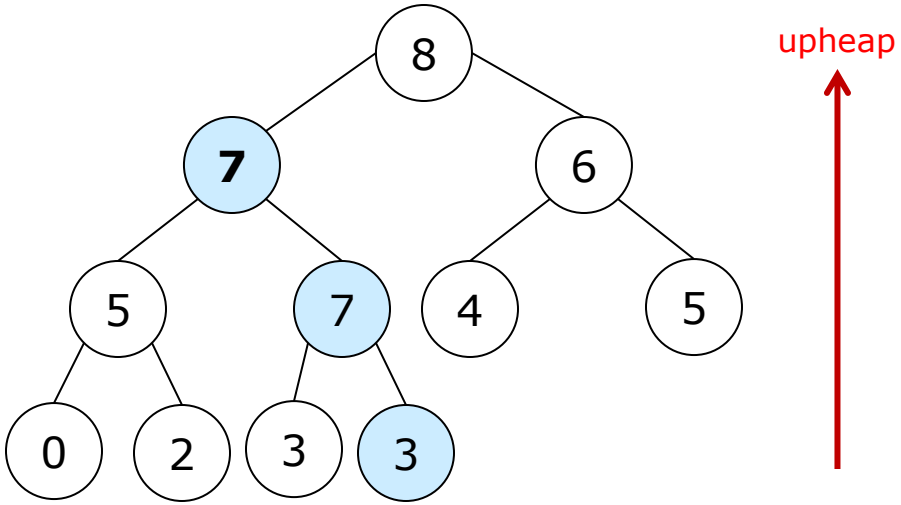


4.3. Heaps and heapsort

- Operations on a heap of *n* elements



Insert 7 into this max-heap?



4.3. Heaps and heapsort

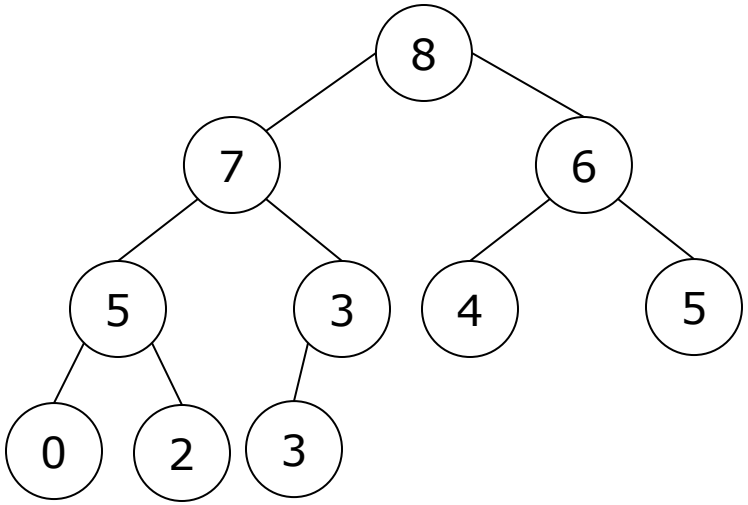
- Downheap on a heap of *n* elements

```
procedure downheap(k: integer);
label  0 ;
var  j, v : integer;
begin
    v:= a[k];
    while k<= n div 2 do
    begin
        j:= 2*k;
        if j < n then if a[j] < a[j+1] then
                        j:=j+1; //a[j] is a larger child
        if v >= a[j] then go to update;
        a[k]:= a[j]; k:= j;
    end;
update: a[k]: =v
end;
```

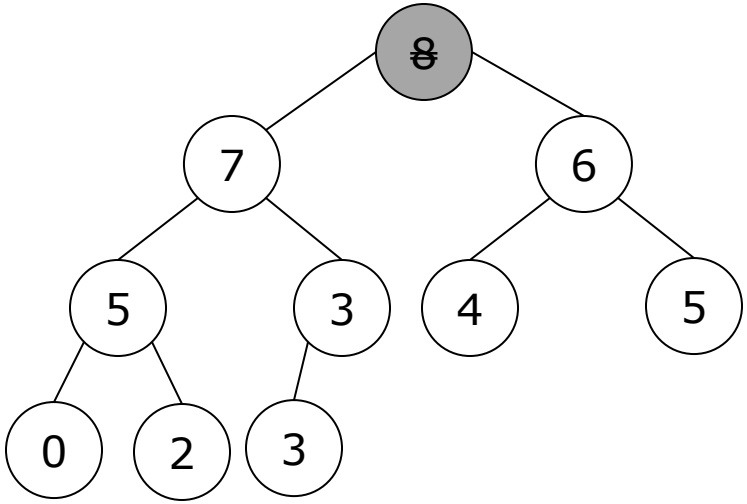
```
function remove: integer;
begin
    remove := a[1];
    a[1] := a[n]; n := n-1;
    downheap(1);
end;
```

4.3. Heaps and heapsort

- Operations on a heap of *n* elements

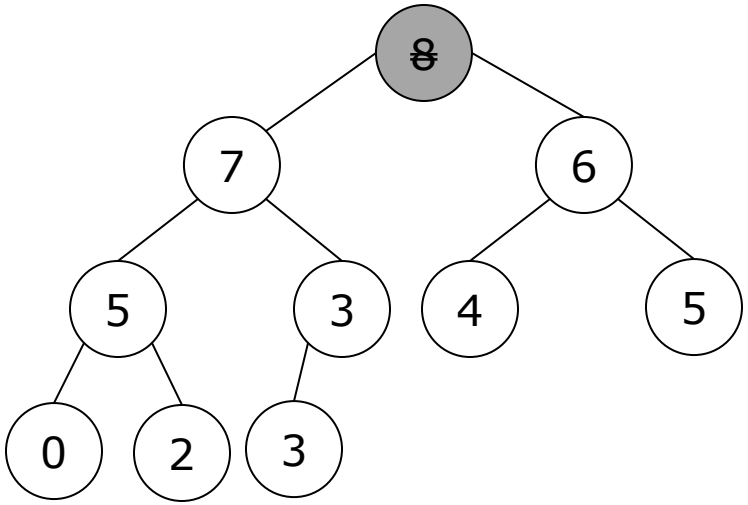


Remove the top-priority element, 8, from this max-heap?

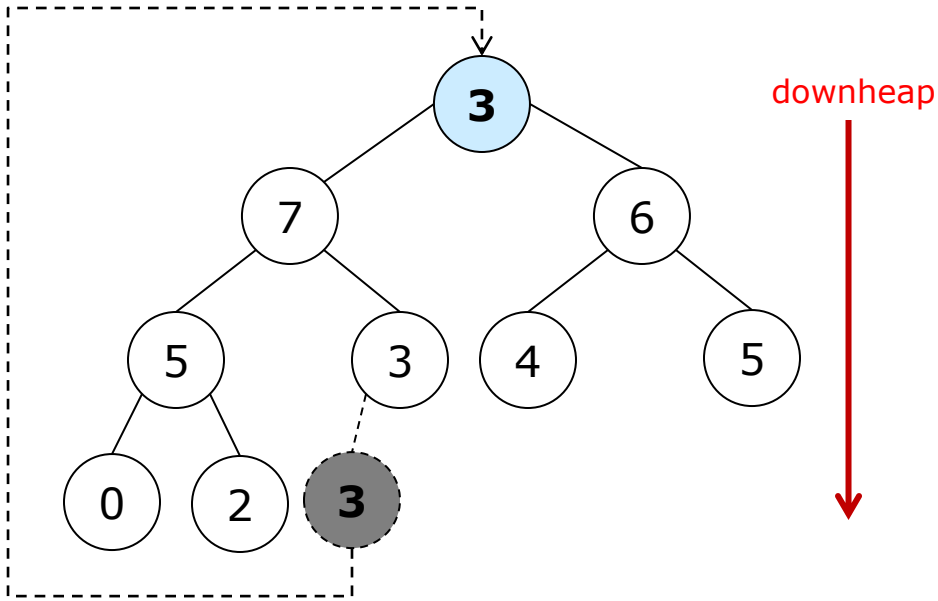


4.3. Heaps and heapsort

- Operations on a heap of *n* elements

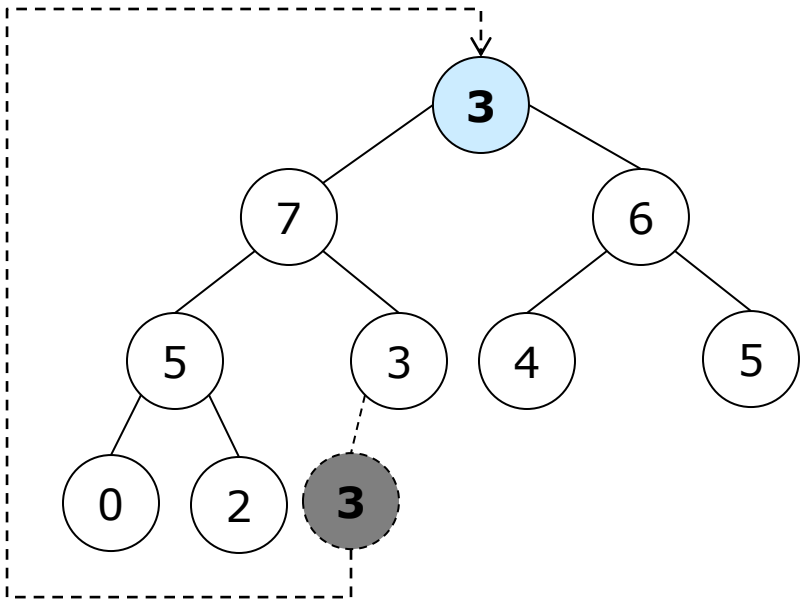


Remove the top-priority element, 8, from this max-heap?

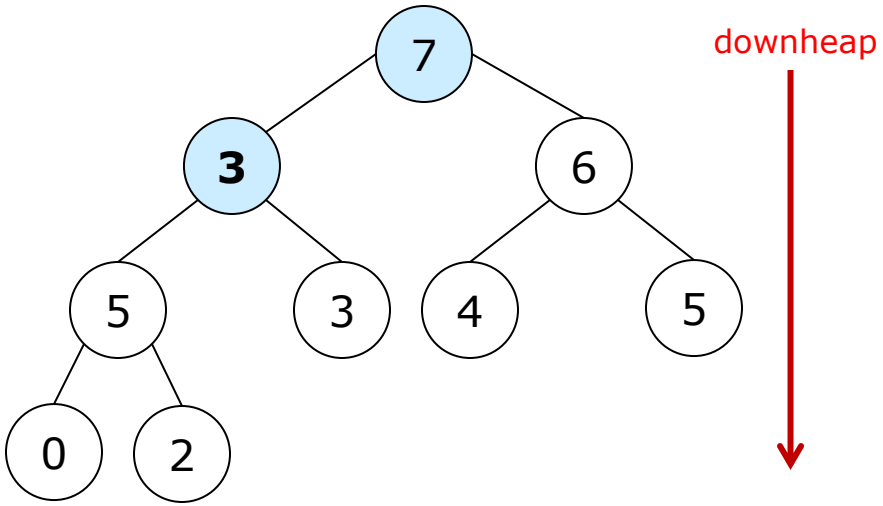


4.3. Heaps and heapsort

- Operations on a heap of *n* elements

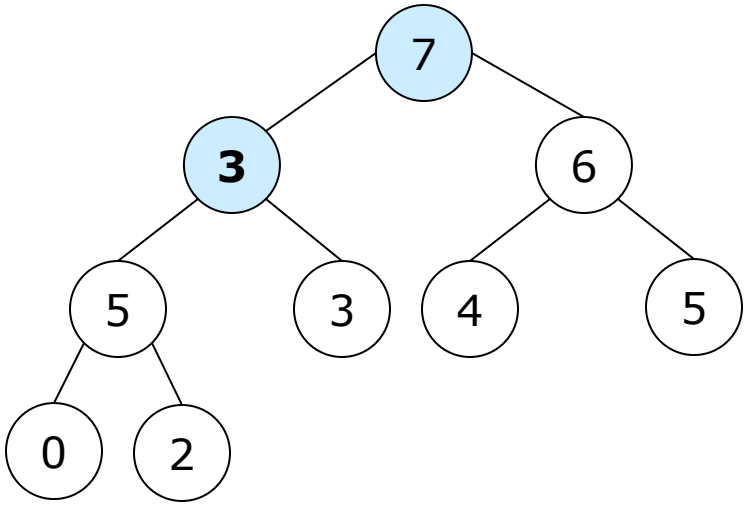


Remove the top-priority element, 8, from this max-heap?

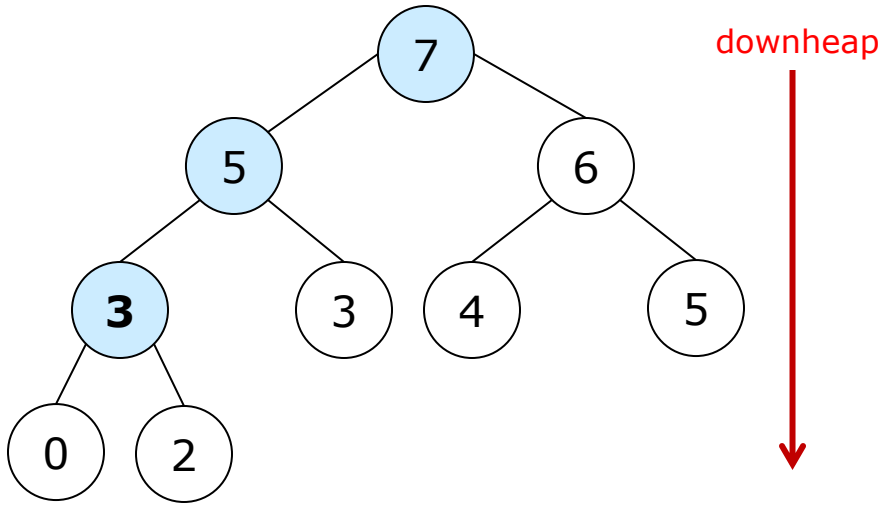


4.3. Heaps and heapsort

- Operations on a heap of *n* elements

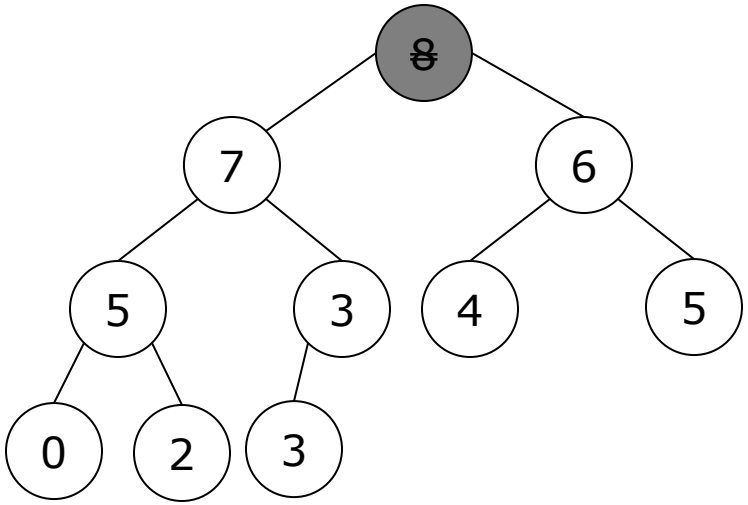


Remove the top-priority element, 8, from this max-heap?

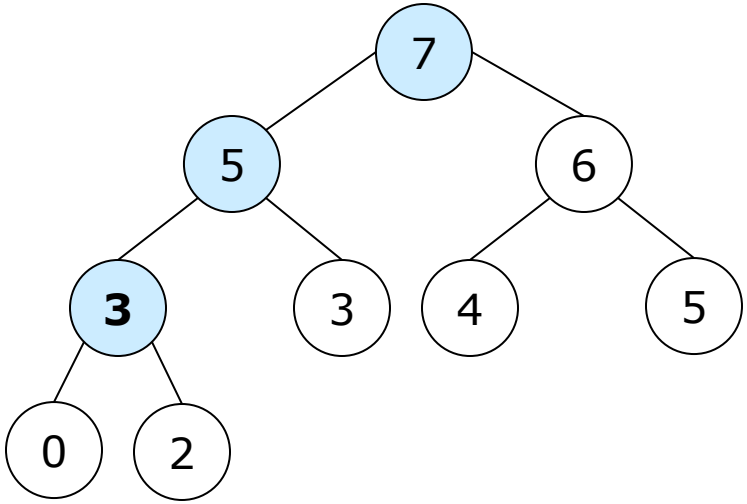


4.3. Heaps and heapsort

- Operations on a heap of *n* elements



Remove the top-priority element, 8, from this max-heap?



4.3. Heaps and heapsort

- Operations on a heap of n elements
  - *Upheap*: insertion of a new element into a heap

    $O(\log_2 n)$ *comparisons*
  - *Downheap*: remove the minimum/maximum (top-priority) element from a heap

    $O(2\log_2 n)$ *comparisons*

4.3. Heaps and heapsort

- Heap construction from *n* elements
  - Top-down heap construction
    - Start with a null complete binary tree (array)
    - Insert each element sequentially into the complete binary tree (array)

upheap ↑

  - Bottom-up heap construction
    - Start with a complete binary tree (array) of all the *n* elements
    - Adjust the complete binary tree (array) to make it a heap
      - From the bottom up to the top
      - From the right to the left

downheap ↓

4.3. Heaps and heapsort

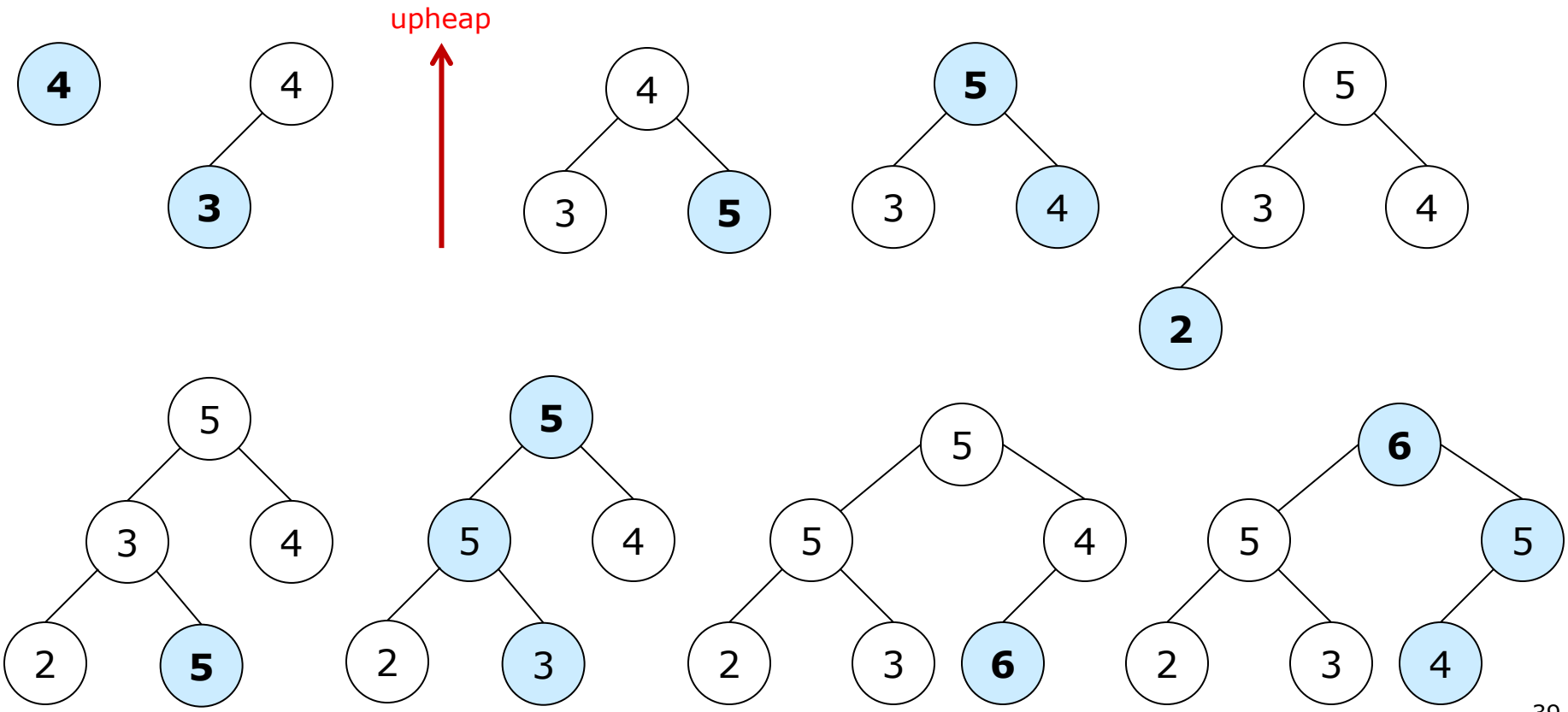
- Top-down heap construction

```
Heap_construction (a, n)

for k:=1 to n do
        insert (a[k]); /* insert a[k] into a heap */
```

4.3. Heaps and heapsort

- Top-down heap construction

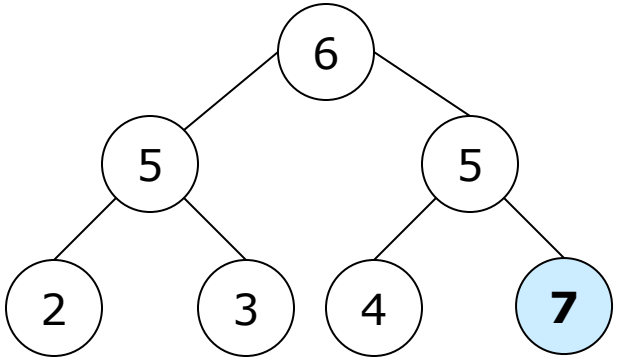
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

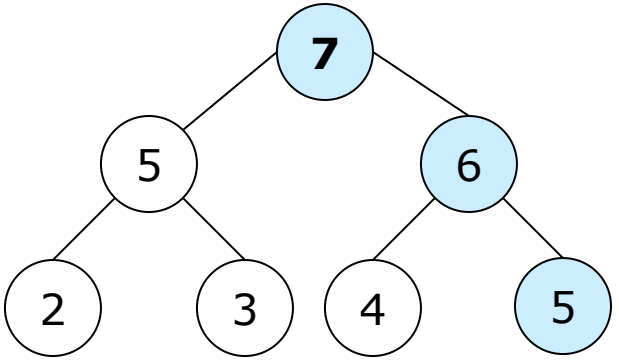


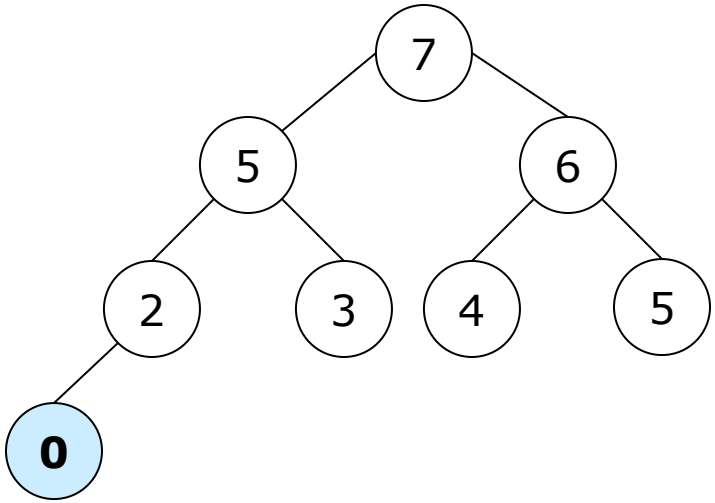
4.3. Heaps and heapsort

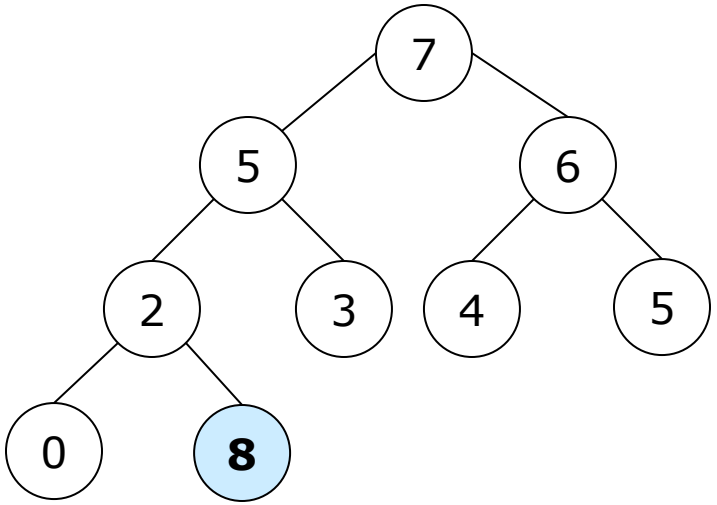
- Top-down heap construction

  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3





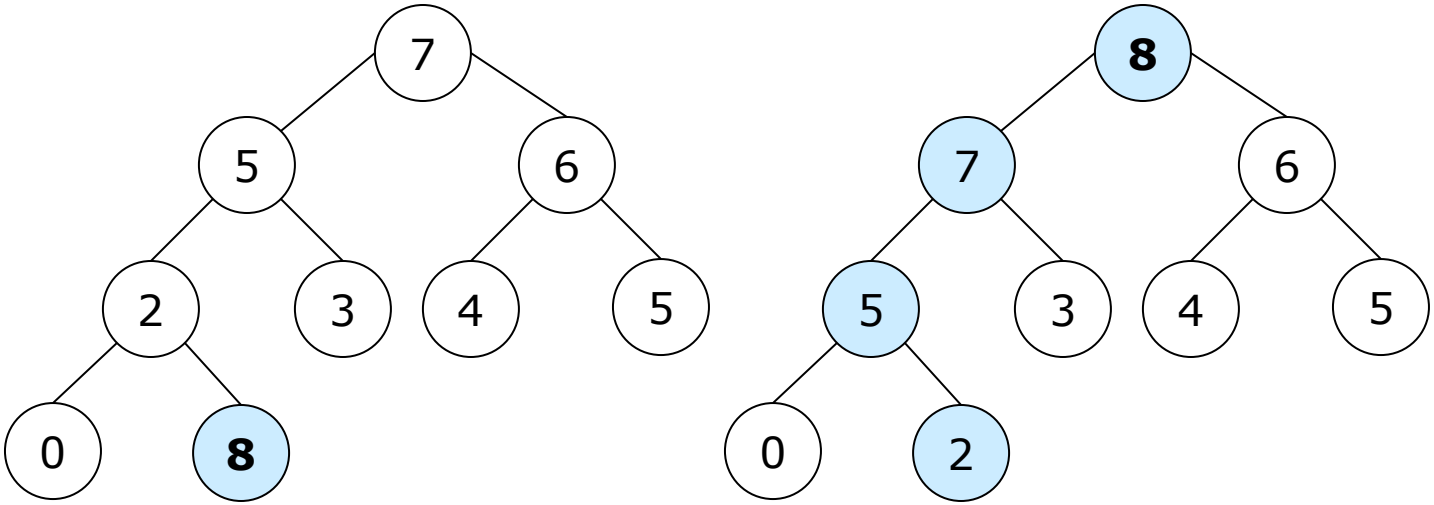




4.3. Heaps and heapsort

- Top-down heap construction

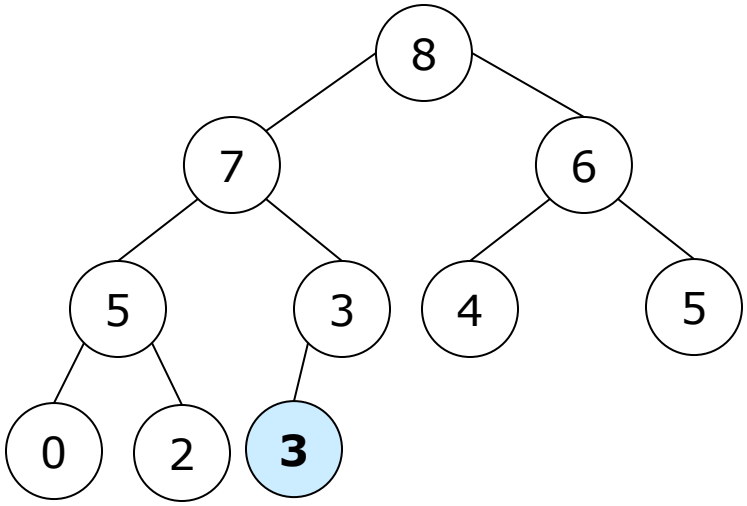
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

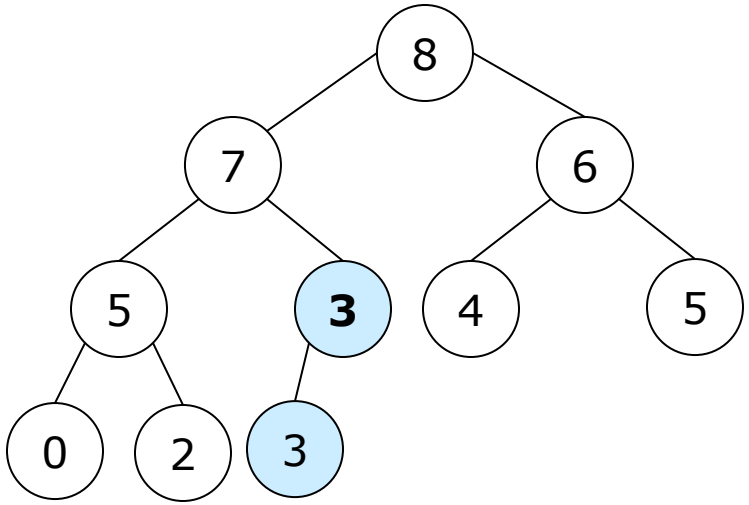


4.3. Heaps and heapsort

- Top-down heap construction

  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

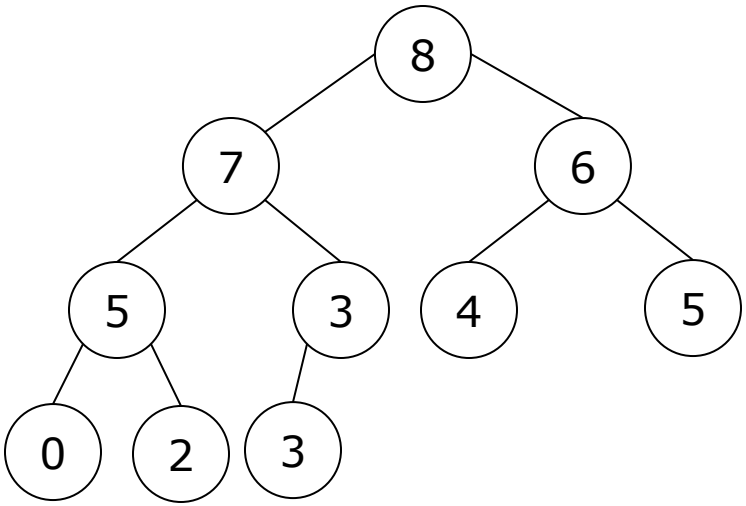




4.3. Heaps and heapsort

- Top-down heap construction

  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

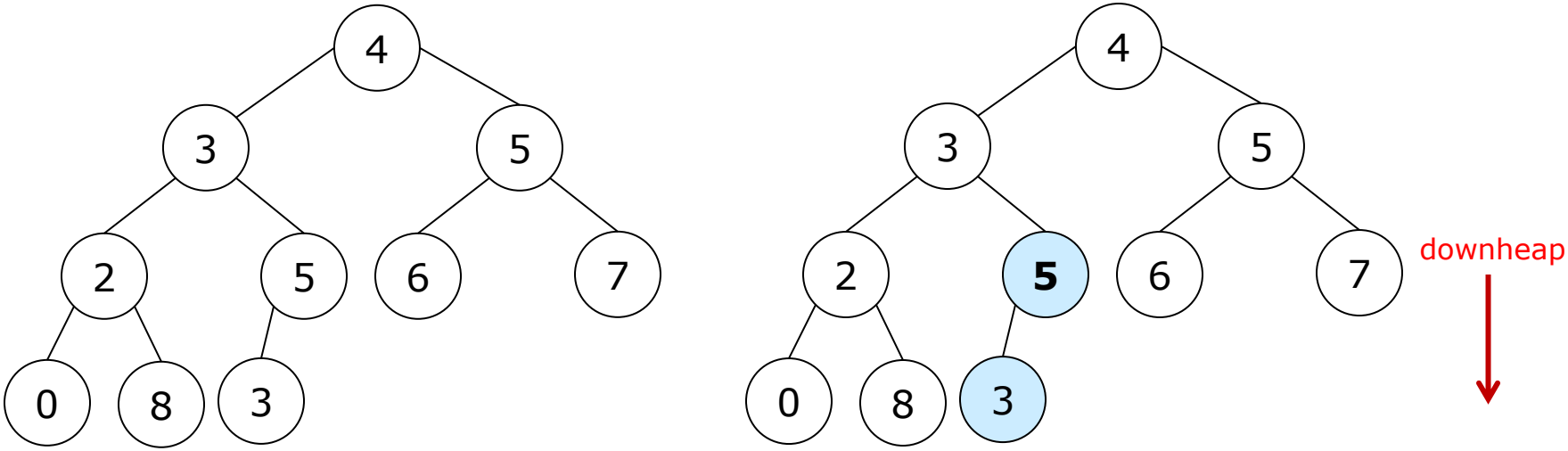
- Bottom-up heap construction

```
Heap_construction (a, n)

for k := n div 2 downto 1 do
        downheap (k);
```

4.3. Heaps and heapsort

- Bottom-up heap construction

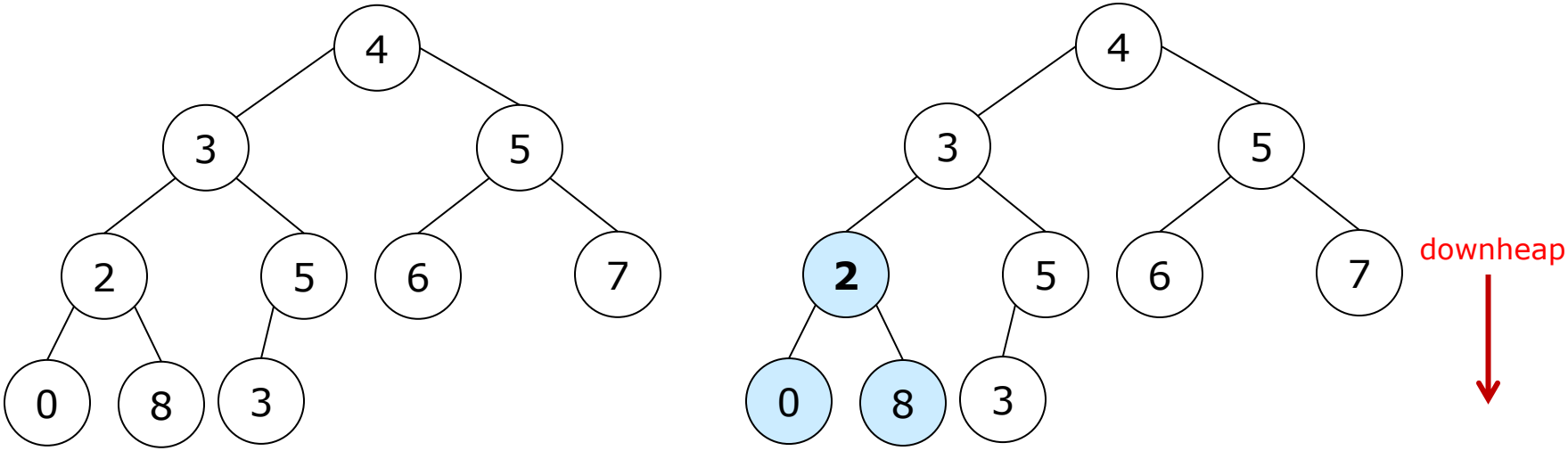
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

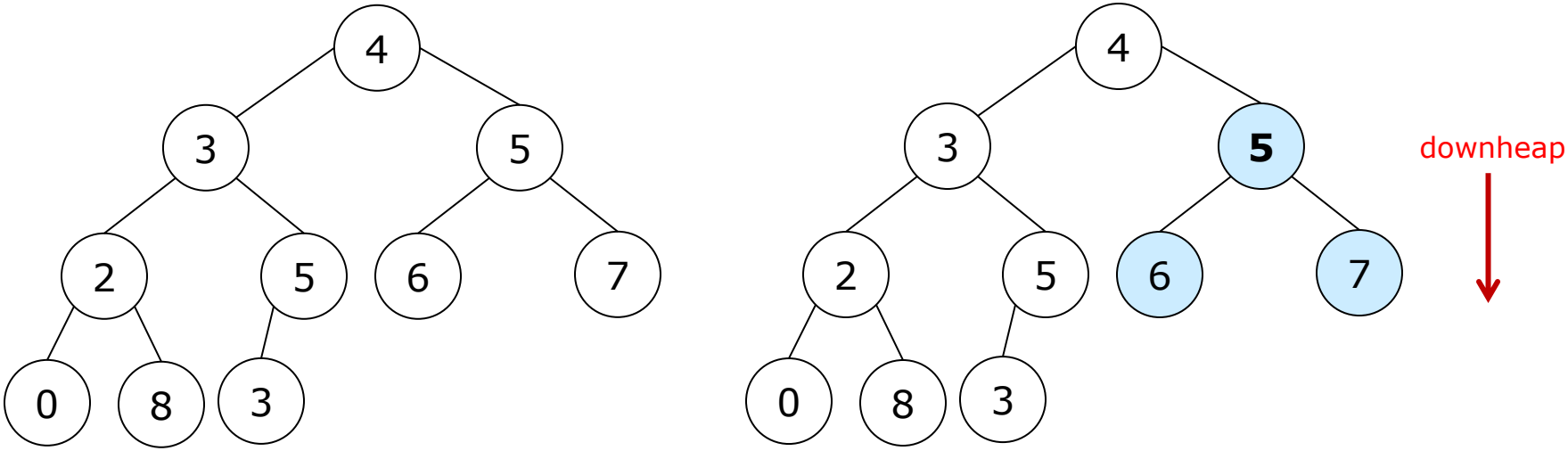
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

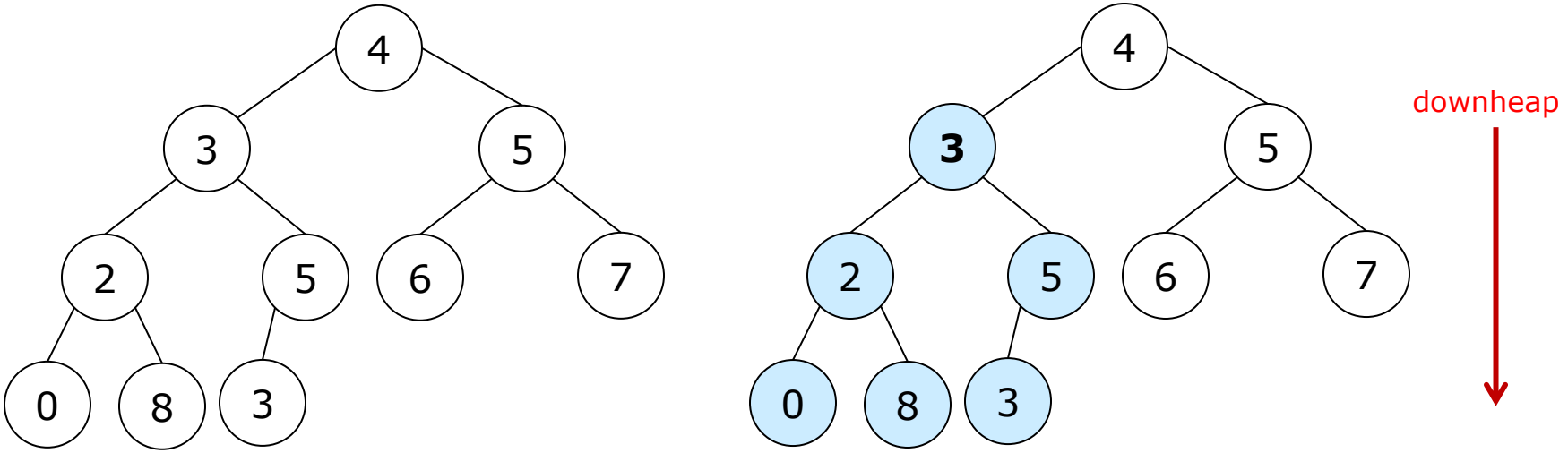
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

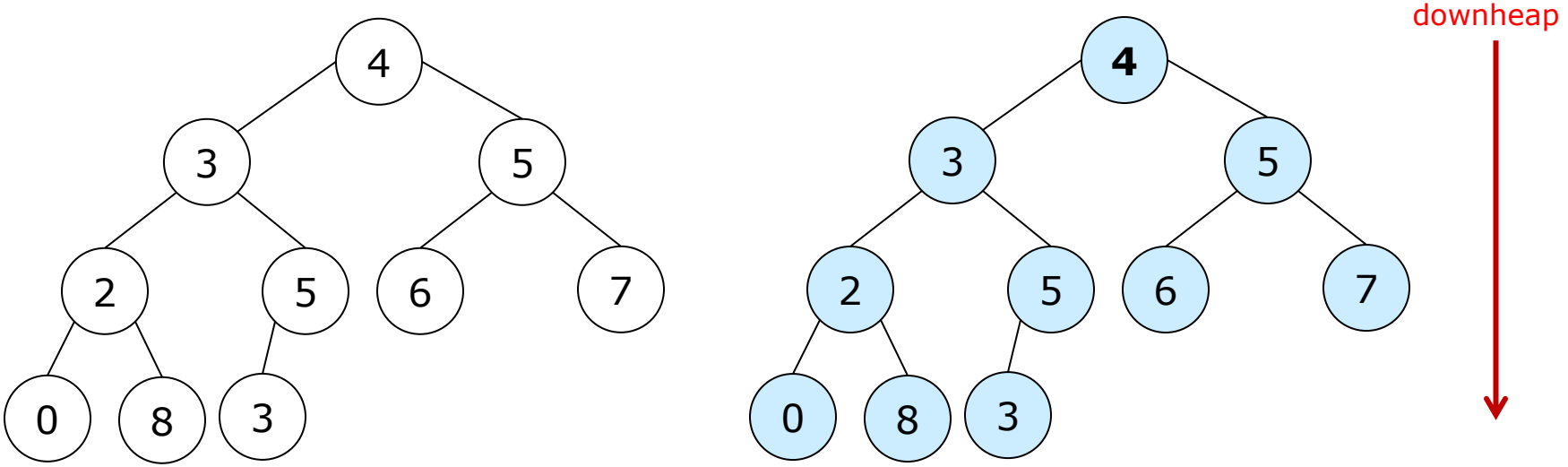
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

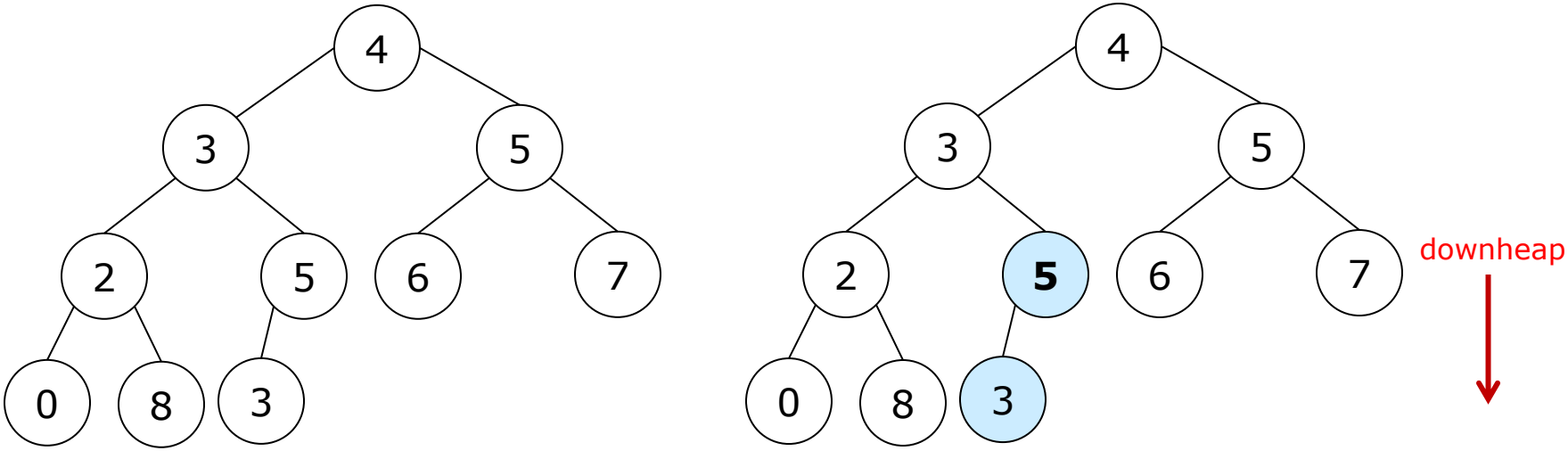
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction – *Let's start …*

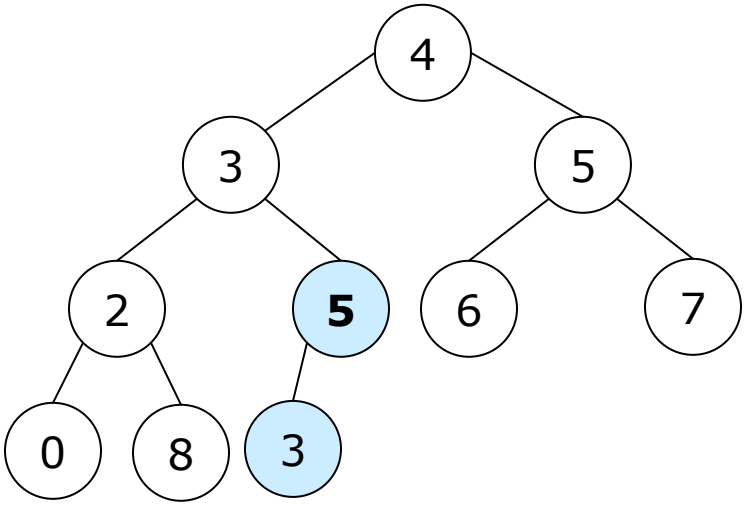
Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

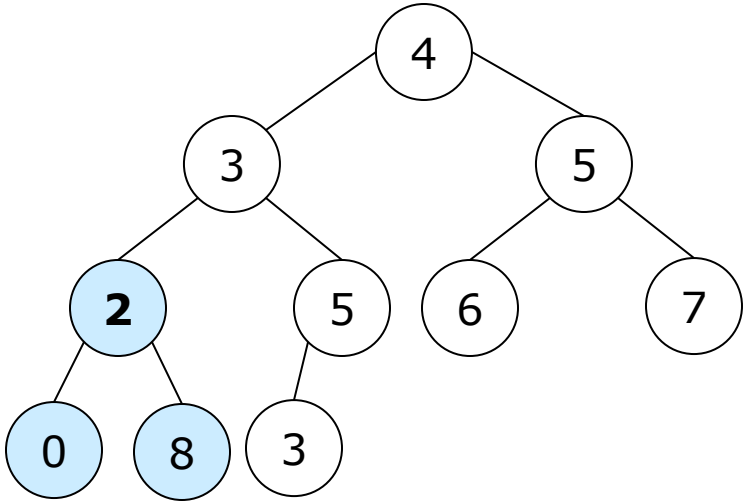


4.3. Heaps and heapsort

- Bottom-up heap construction

  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

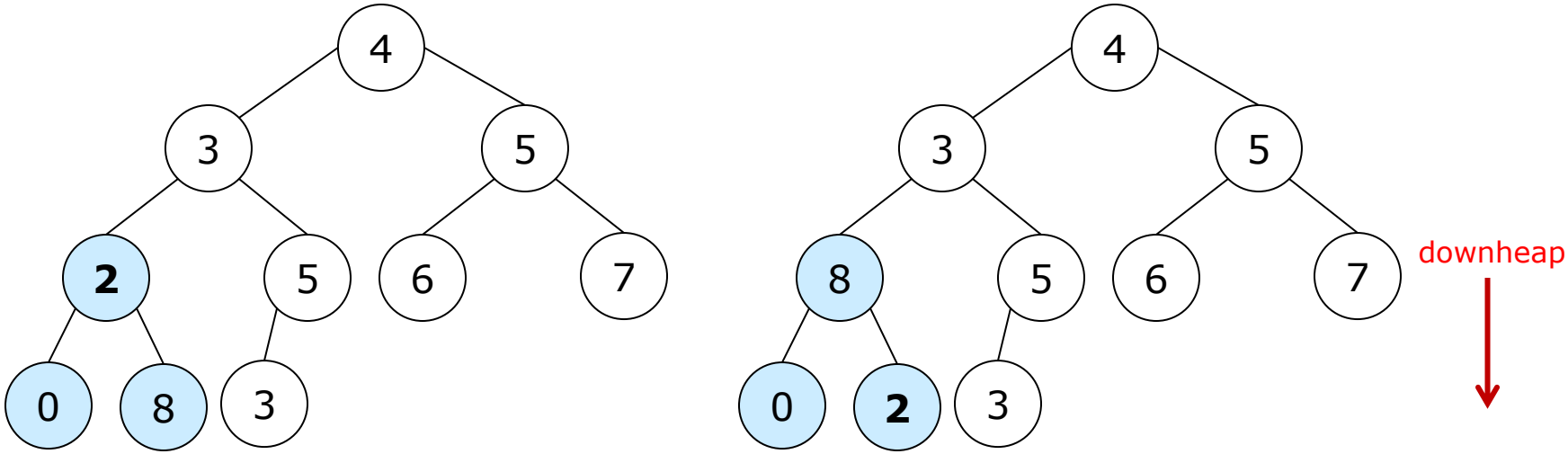




4.3. Heaps and heapsort

- Bottom-up heap construction

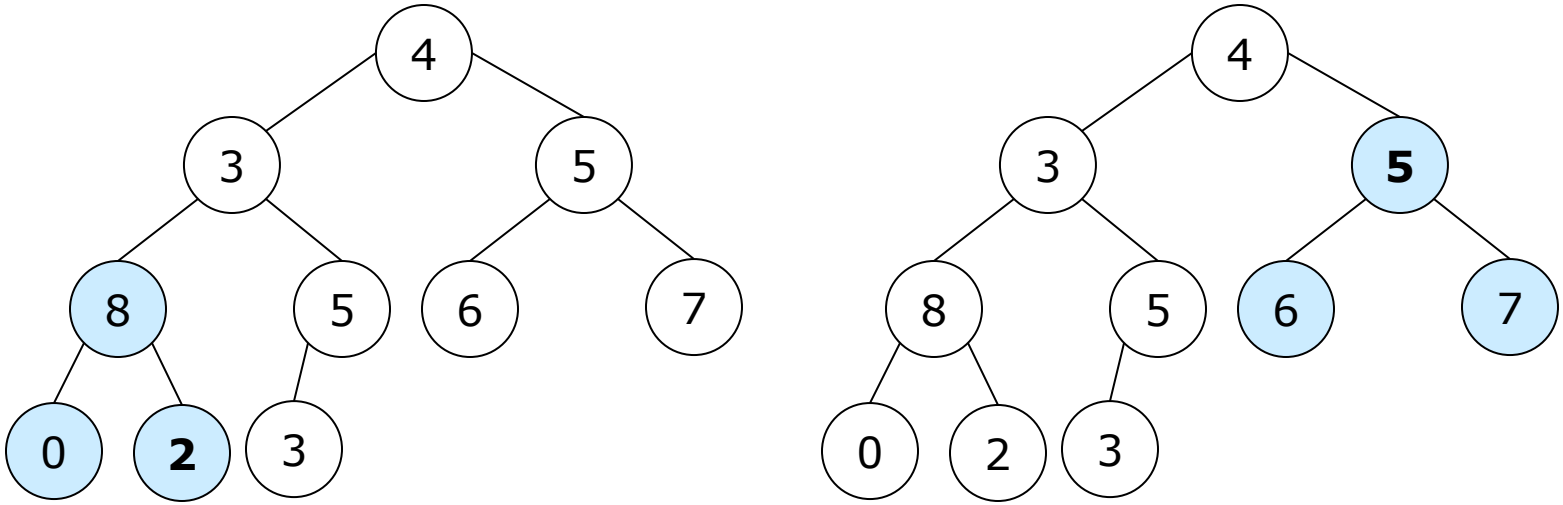
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

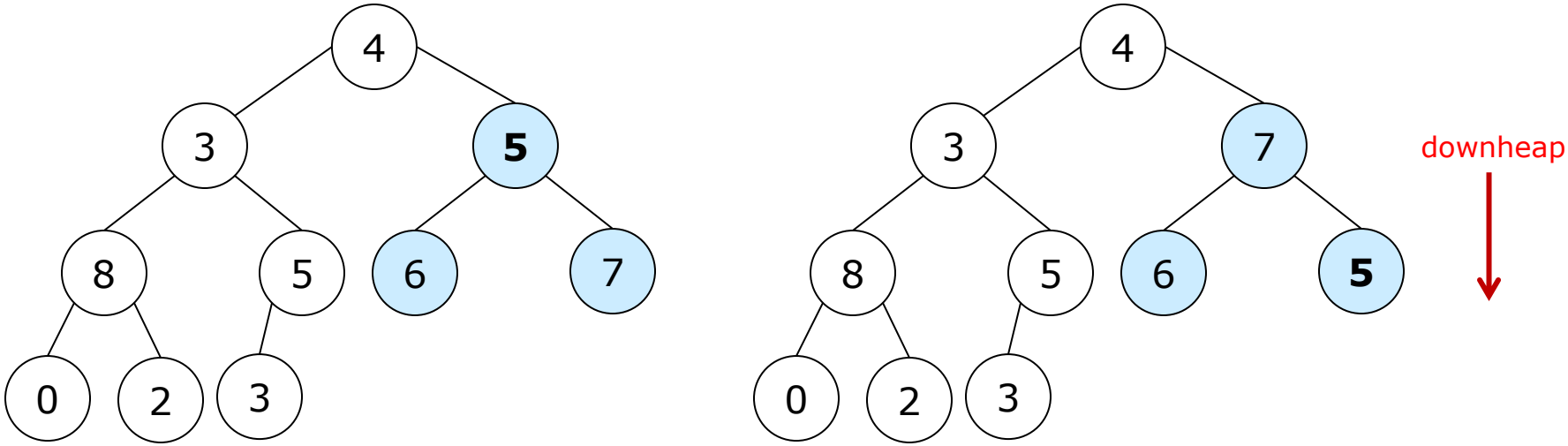
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

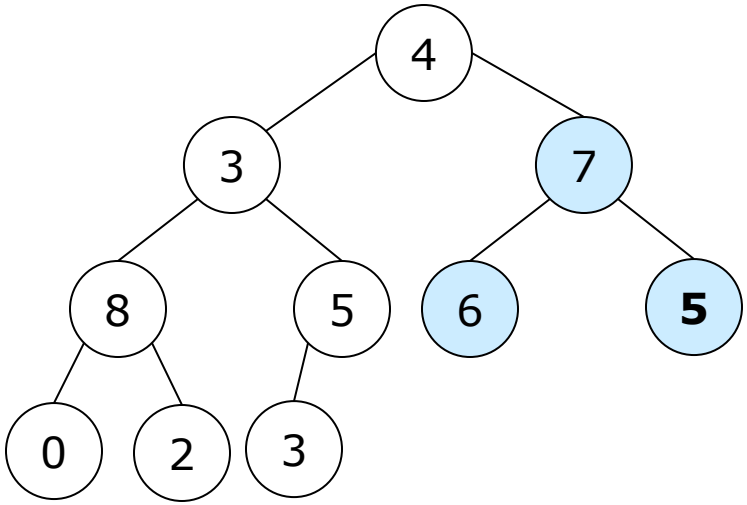
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

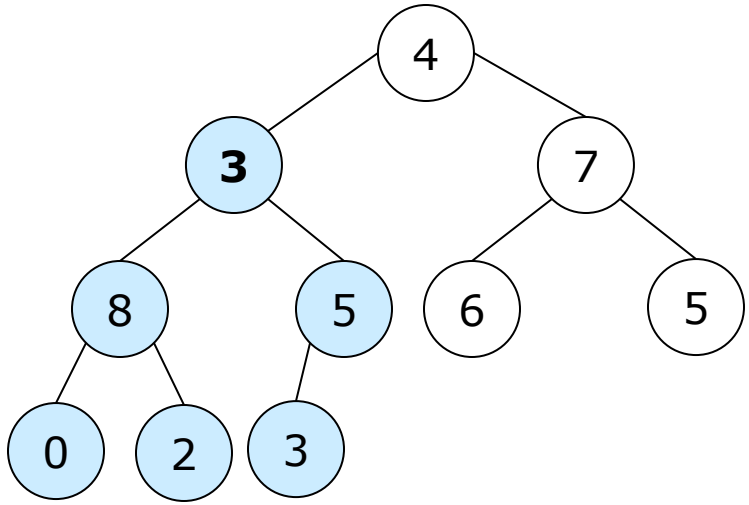


4.3. Heaps and heapsort

- Bottom-up heap construction

  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3

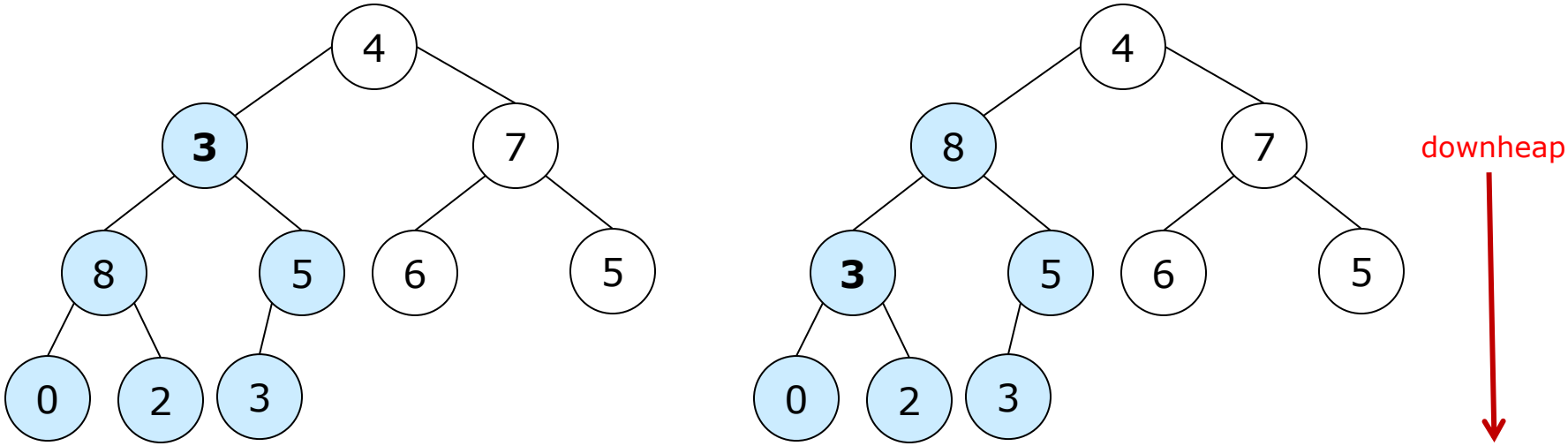




4.3. Heaps and heapsort

- Bottom-up heap construction

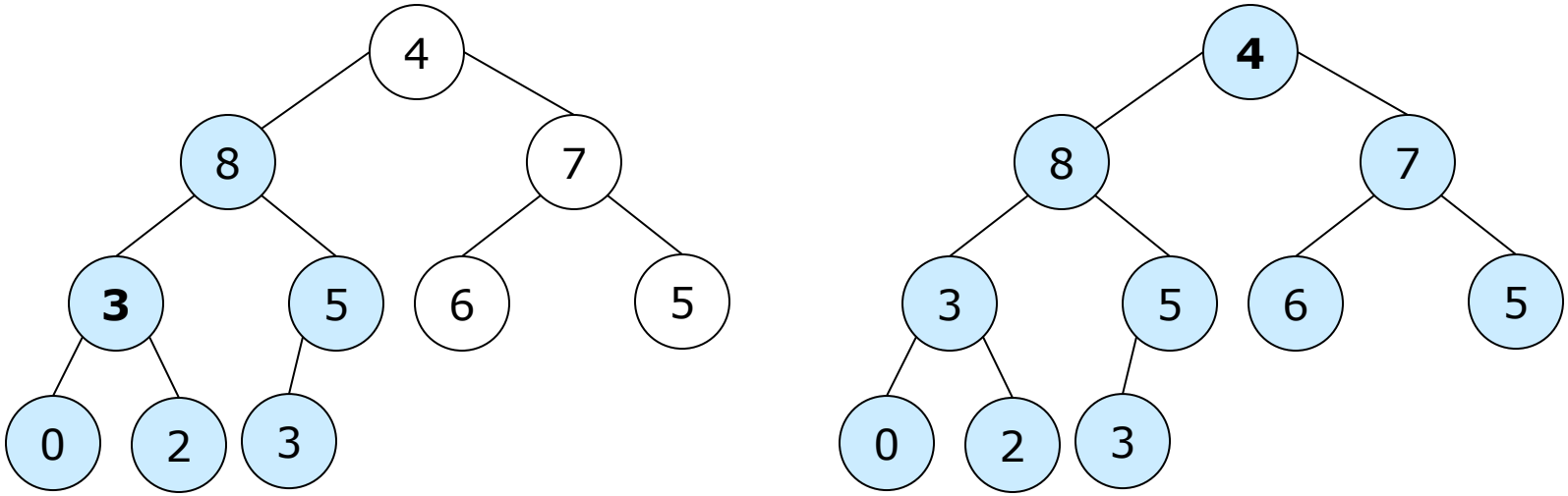
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

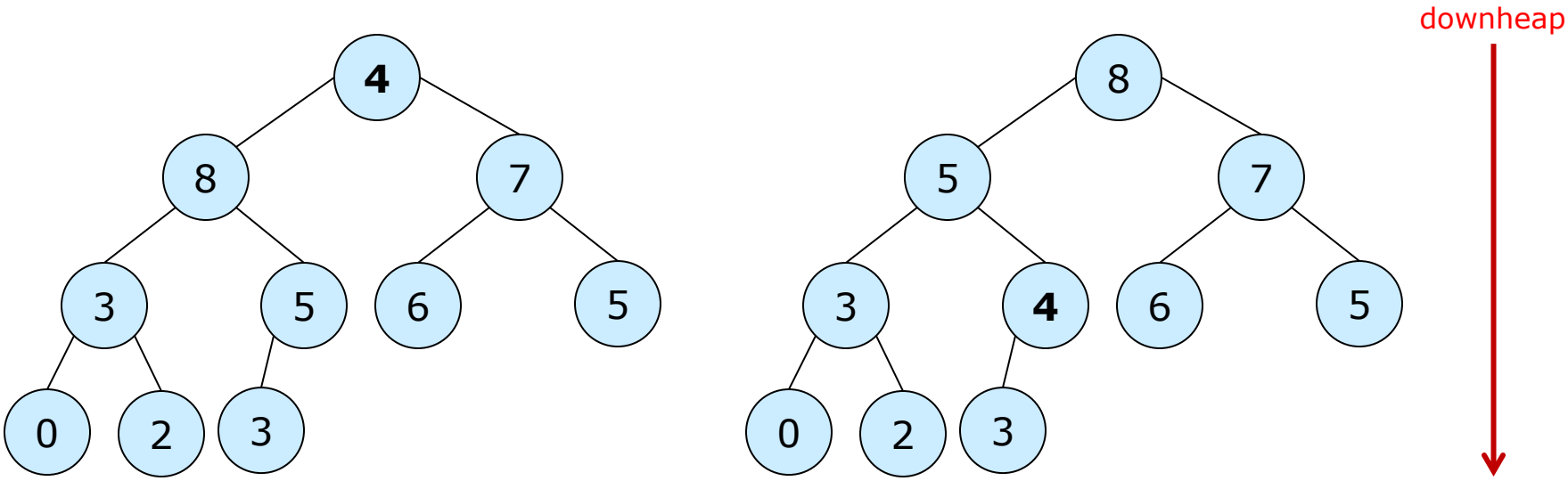
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

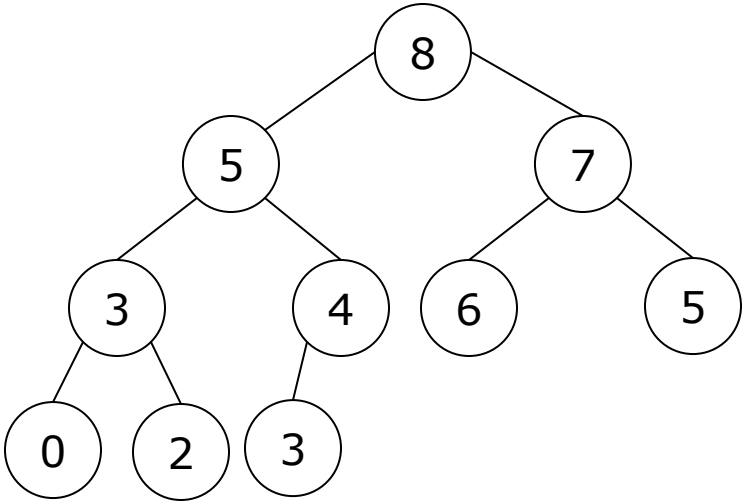
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



4.3. Heaps and heapsort

- Bottom-up heap construction

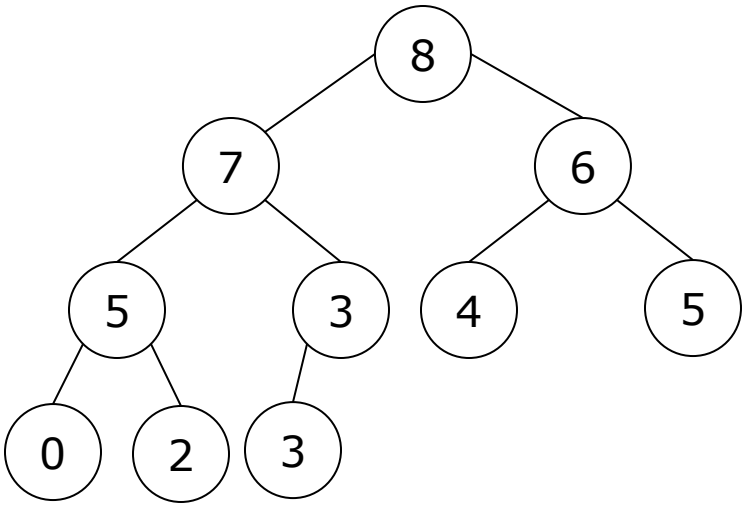
  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



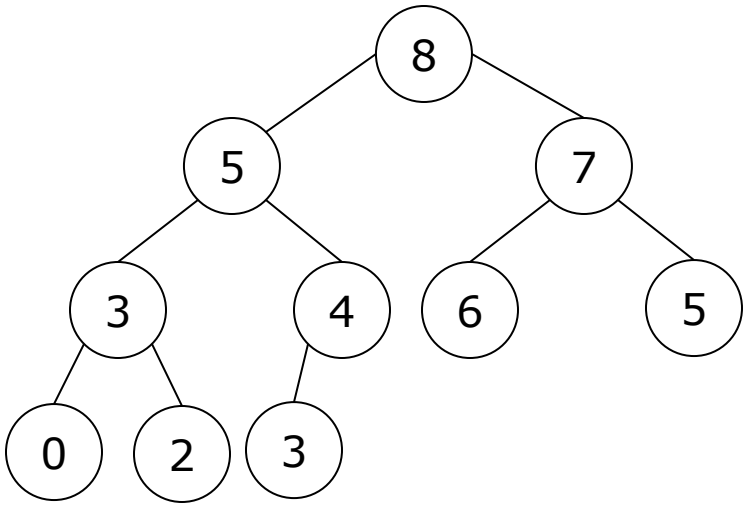
4.3. Heaps and heapsort

- Heap construction

  Max-heap: 4, 3, 5, 2, 5, 6, 7, 0, 8, 3



Top-down



Bottom-up

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With top-down heap construction

$$C_n = O(n \log_2 n)$$

  - With bottom-up heap construction

$$C_n = O(n)$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With top-down heap construction

$$C_n = O(n\log_2 n)$$

```
for k:=1 to n do
        insert (a[k]); /* insert a[k] into a heap */
```

$$C_n = log_2 2 + \cdots + log_2(n-2) + log_2(n-1) + log_2 n$$
$$C_n \leq log_2 n + \cdots + log_2 n + log_2 n + log_2 n$$
$$C_n \leq n log_2 n = O(n log_2 n)$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With top-down heap construction

$$C_n = O(n\log_2 n)$$

for *k*:=1 **to** *n* **do**

insert (*a*[*k*]); /* insert *a*[*k*] into a heap */

$$C_n = log_2 2 + \cdots + log_2(n-2) + log_2(n-1) + log_2 n$$
$$C_n \leq log_2 n + \cdots + log_2 n + log_2 n + log_2 n$$
$$C_n \leq nlog_2 n = O(nlog_2 n)$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With bottom-up heap construction

$$C_n = O(n)$$

```
for k := n div 2 downto 1 do
        downheap (k)
```

$$C_n = \left(\frac{n}{2.2}\right).2.log_2 2 + \cdots + 4.2.log_2(n/4) + 2.2.log_2(n/2) + 1.2.log_2 n$$

$$C_n \leq 2n = O(n)$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With bottom-up heap construction

$$C_n = O(n)$$

for *k* := *n* div 2 **downto** 1 **do**

downheap (*k*);

#subheap #cmps Height of subheap

$$C_n = \left(\frac{n}{2.2}\right).2.log_2 2 + \cdots + 4.2.log_2(n/4) + 2.2.log_2(n/2) + 1.2.log_2 n$$

$$C_n \leq 2n = O(n)$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With bottom-up heap construction

$$C_n = O(n)$$

$$C_n = \left(\frac{n}{2.2}\right).2.log_2 2 + \cdots + 4.2.log_2\left(\frac{n}{4}\right) + 2.2.log_2\left(\frac{n}{2}\right) + 1.2.log_2 n$$

Suppose: $n = 2^h$, C_n is rewritten as:

$$C_{2^h} = 2^{h-2}.2.1 + 2^{h-3}.2.2 + \ \ldots + 2^2.2.(h-2) + 2.2.(h-1) + 1.2.h$$

$$2C_{2^h} = 2^{h-1}.2.1 + 2^{h-2}.2.2 + \ \ldots + 2^3.2.(h-2) + 2^2.2.(h-1) + 2.2.h$$

$$\rightarrow C_{2^h} = 2C_{2^h} - C_{2^h}$$

$$C_{2^h} = 2^{h-1}.2.1 + 2^{h-2}.2.1 + \ \ldots + 2^2.2.1 \ + 2.2.1 - 1.2.h$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With bottom-up heap construction

$$C_n = O(n)$$

$$C_n = \left(\frac{n}{2.2}\right).2.log_2 2 + \cdots + 4.2.log_2\left(\frac{n}{4}\right) + 2.2.log_2\left(\frac{n}{2}\right) + 1.2.log_2 n$$

Suppose: $n = 2^h$, C_n is rewritten as:

$$C_{2^h} = 2^{h-2}.2.1 + 2^{h-3}.2.2 + \ \ldots + 2^2.2.(h-2) + 2.2.(h-1) + 1.2.h$$

$$C_{2^h} = 2^{h-1}.2.1 + 2^{h-2}.2.1 + \ \ldots + 2^2.2.1 \ + 2.2.1 - 1.2.h$$

$$C_{2^h} = 2(2^{h-1} + 2^{h-2} + \ \ldots + 2^2 \ + 2 + 1 - 1) - 1.2.h$$

$$C_{2^h} = 2(2^h - 1 - 1) - 1.2.h = 2.2^h - 2.h - 4$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With bottom-up heap construction

$$C_n = O(n)$$

$$C_n = \left(\frac{n}{2.2}\right).2.log_2 2 + \cdots + 4.2.log_2\left(\frac{n}{4}\right) + 2.2.log_2\left(\frac{n}{2}\right) + 1.2.log_2 n$$

Suppose: $n = 2^h$, C_n is rewritten as:

$$C_{2^h} = 2^{h-2}.2.1 + 2^{h-3}.2.2 + \ \ldots + 2^2.2.(h-2) + 2.2.(h-1) + 1.2.h$$

$$C_{2^h} = 2(2^h - 1 - 1) - 1.2.h = 2.2^h - 2.h - 4$$

$$\rightarrow C_n = 2n - 2log_2 n - 4 \leq 2n = O(n)$$

4.3. Heaps and heapsort

- Time complexity of heap construction for a heap of *n* elements
  - Abstract operation: *comparison*
  - With bottom-up heap construction

$$C_n = O(n)$$

```
for k := n div 2 downto 1 do
        downheap (k);
```

$$C_n = \left(\frac{n}{2.2}\right).2.log_2 2 + \cdots + 4.2.log_2(n/4) + 2.2.log_2(n/2) + 1.2.log_2 n$$

$$C_n \leq 2n = O(n)$$

4.3. Heaps and heapsort

- Heapsort
  - Task: sort *n* elements in ascending order
  - Steps:
    - Build a max-heap
    - Remove the maximum element from the heap one by one
    - Return *n* elements in ascending order
  - Example: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3

    ➔ 0, 2, 3, 3, 4, 5, 5, 6, 7, 8

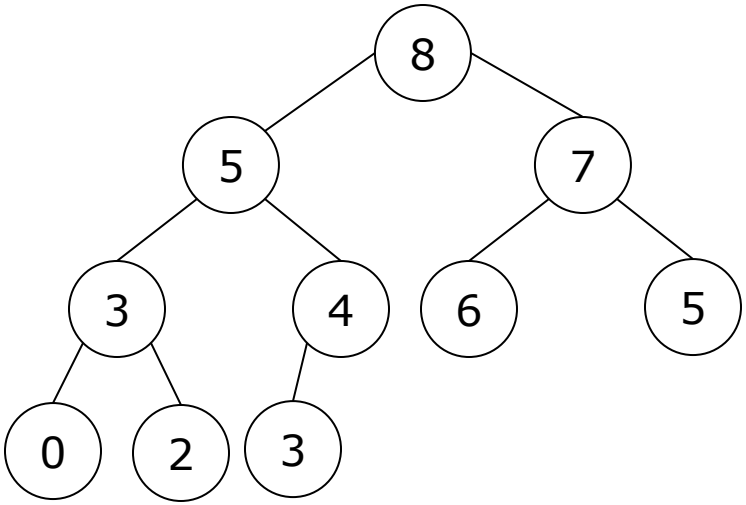
4.3. Heaps and heapsort

- Heapsort
  - Task: sort *n* elements in ascending order
  - Steps:
    - Build a max-heap
    - Remove the maximum element from the heap one by one
    - Return *n* elements in ascending order

```
Heap_construction (a, n);
for k := n downto 1 do
    a[k]:= remove; /*putting the removed element into array a*/
```

4.3. Heaps and heapsort

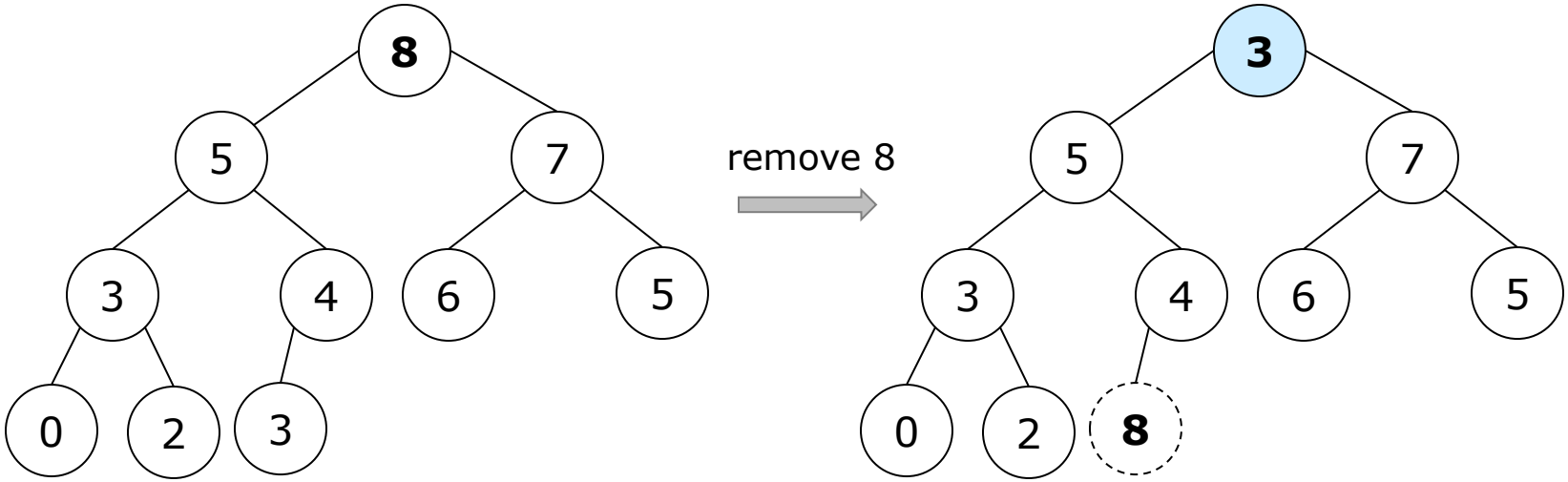
- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



A max-heap of *n* given elements

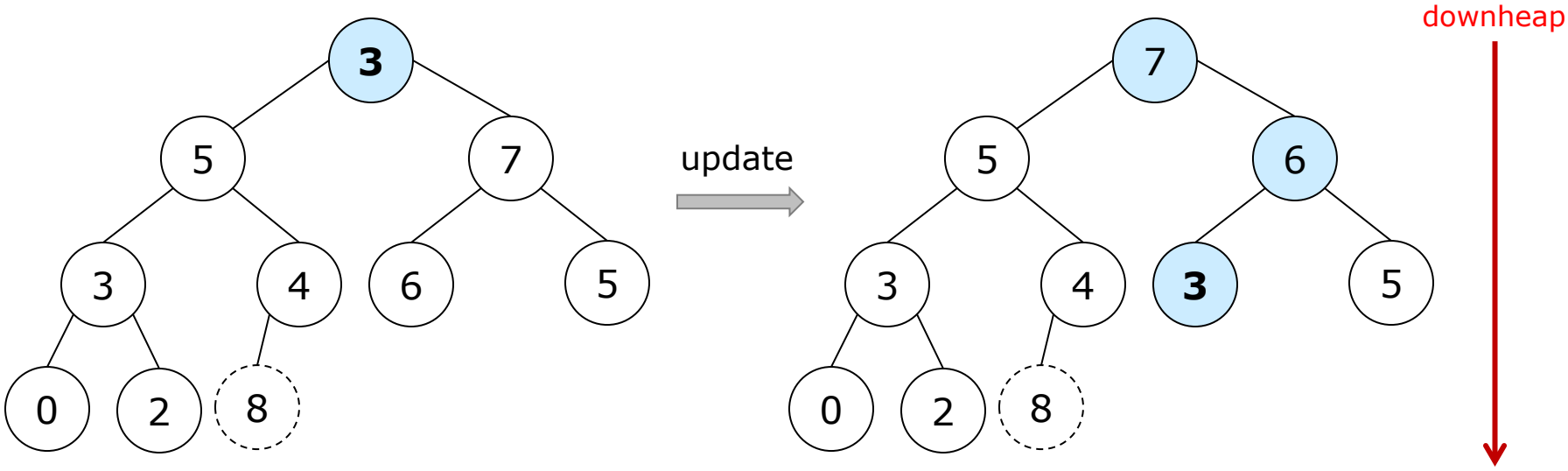
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



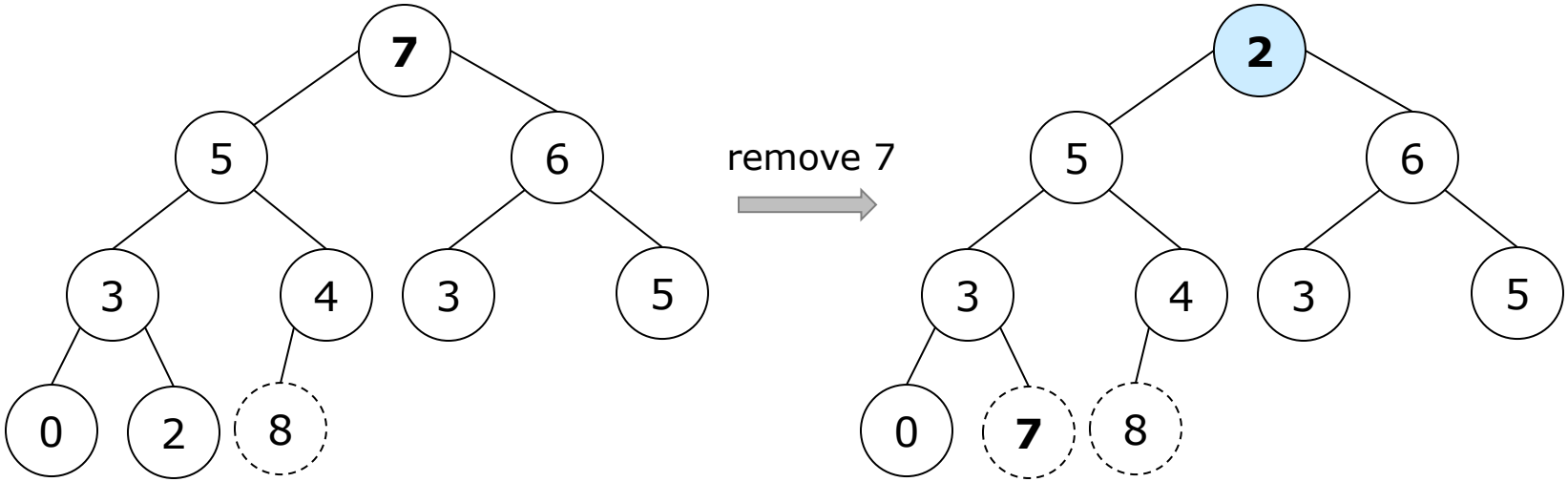
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



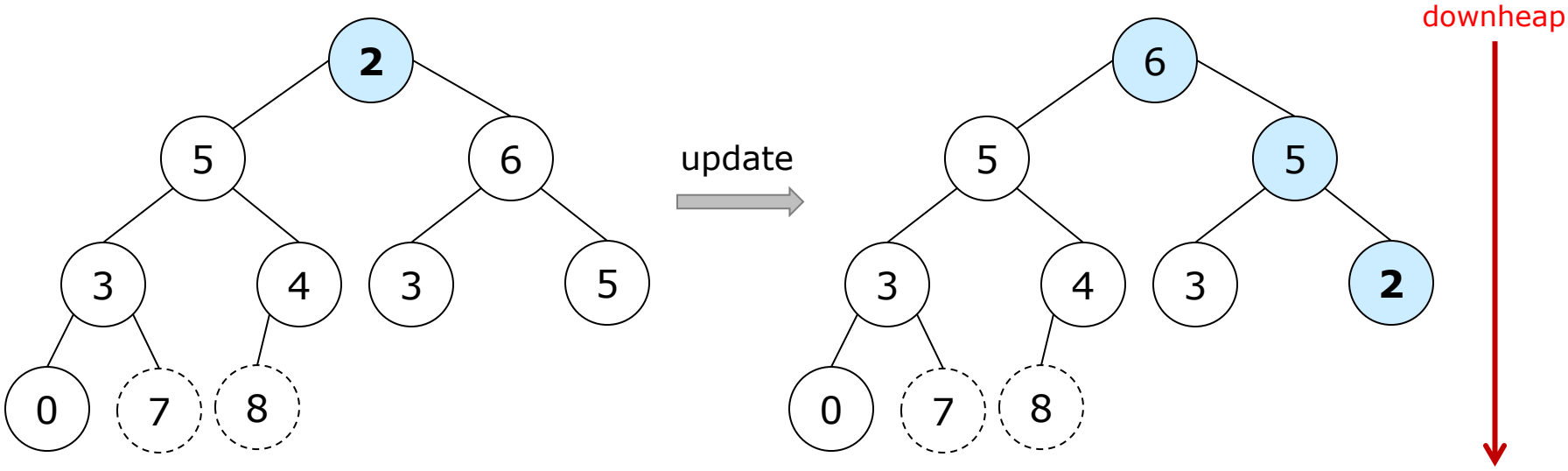
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



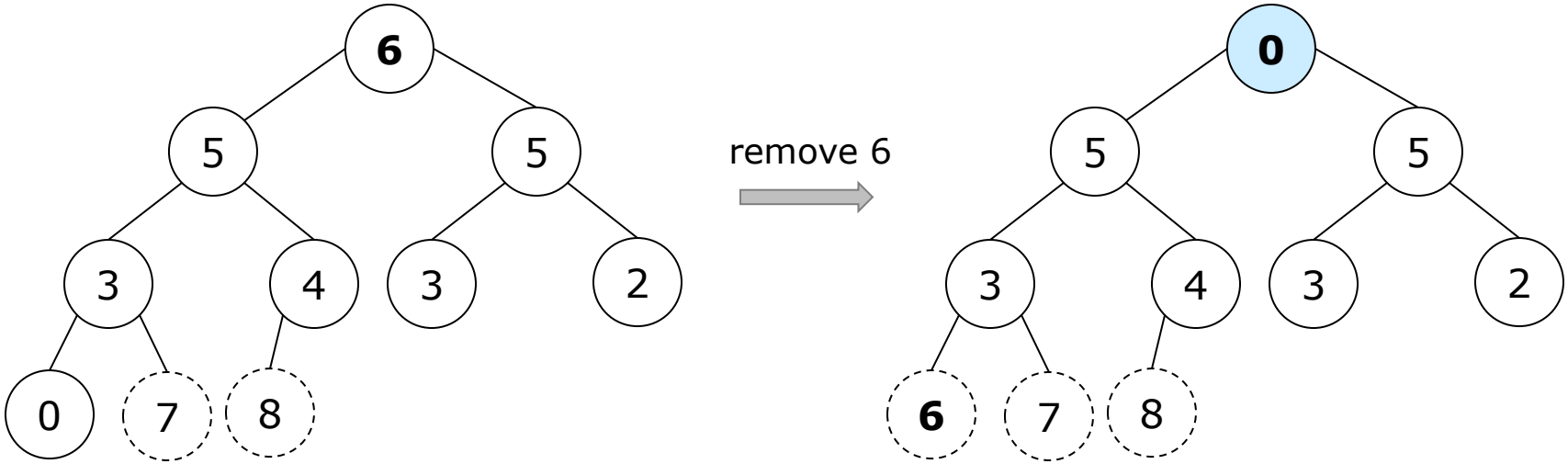
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



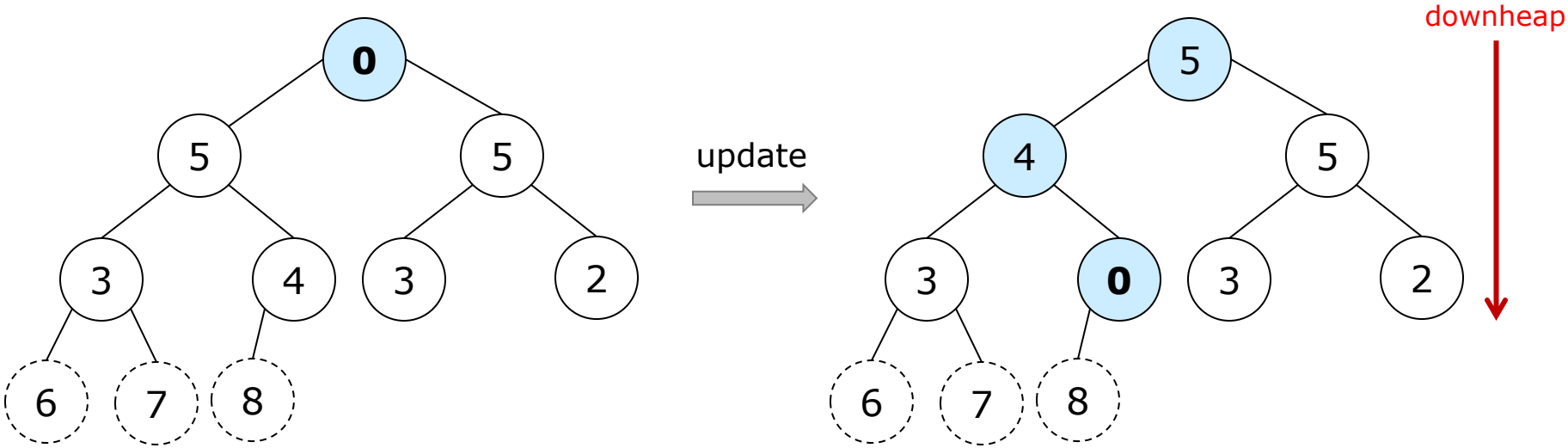
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



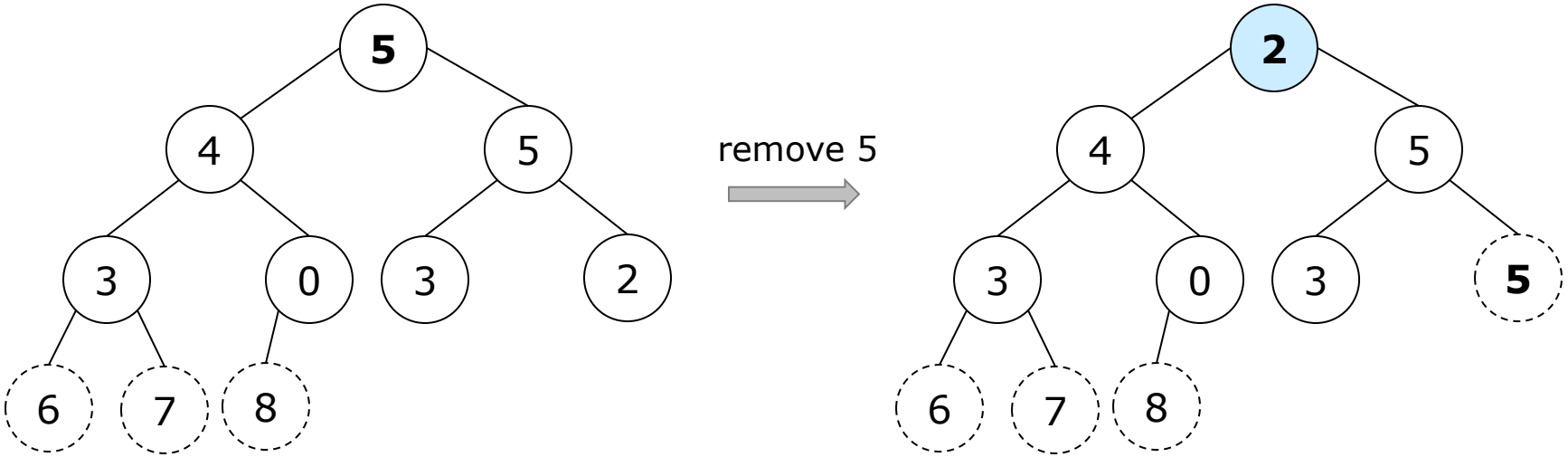
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



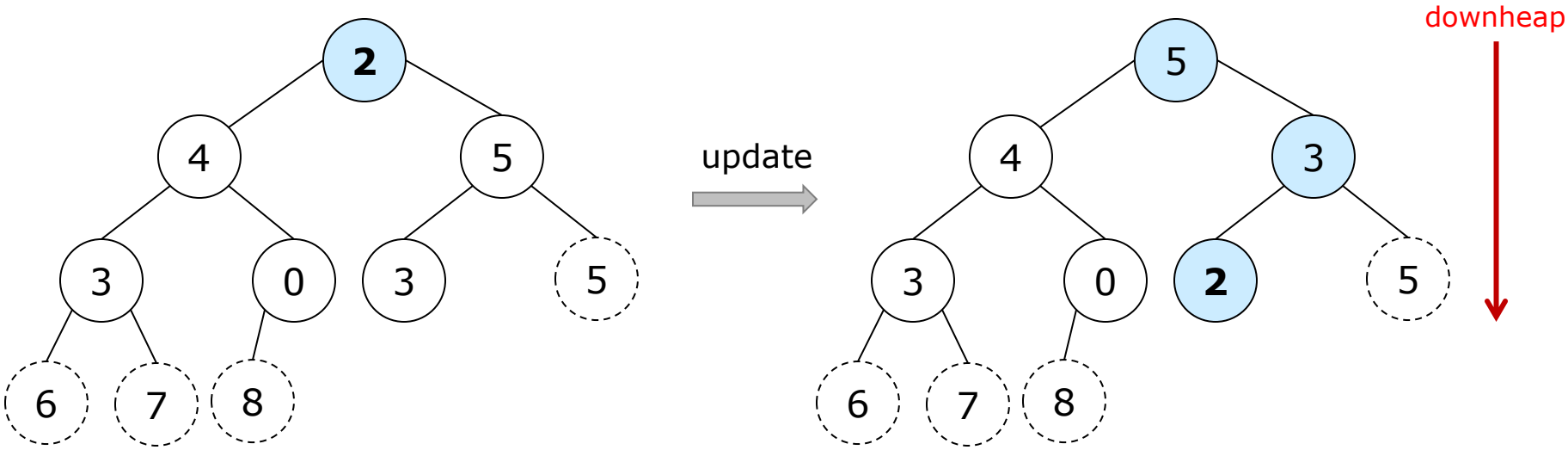
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



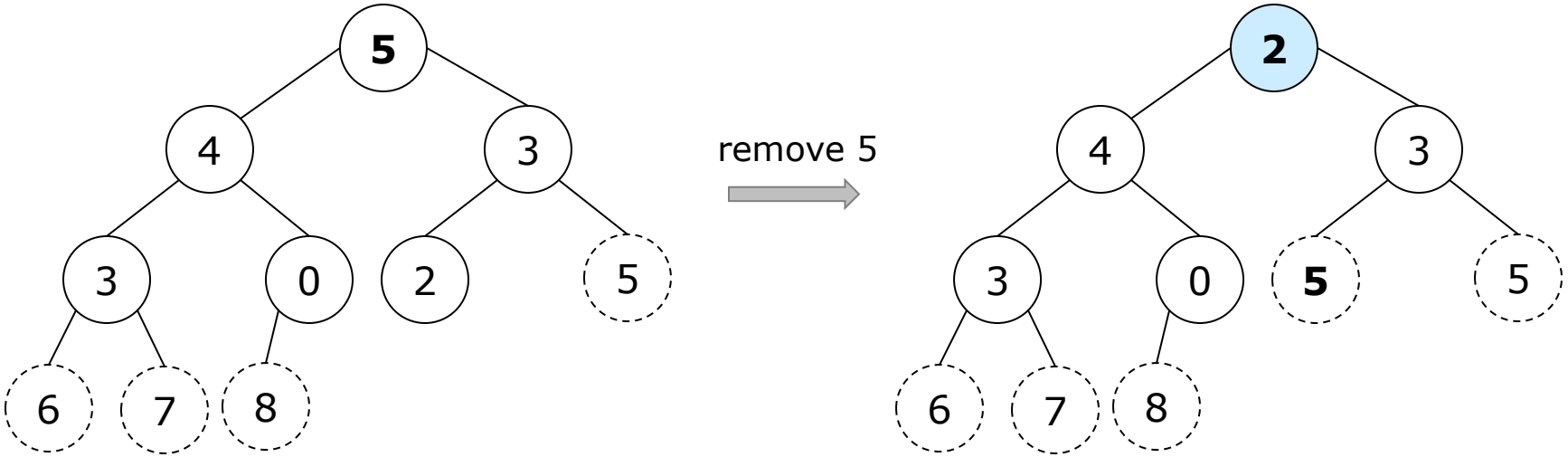
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



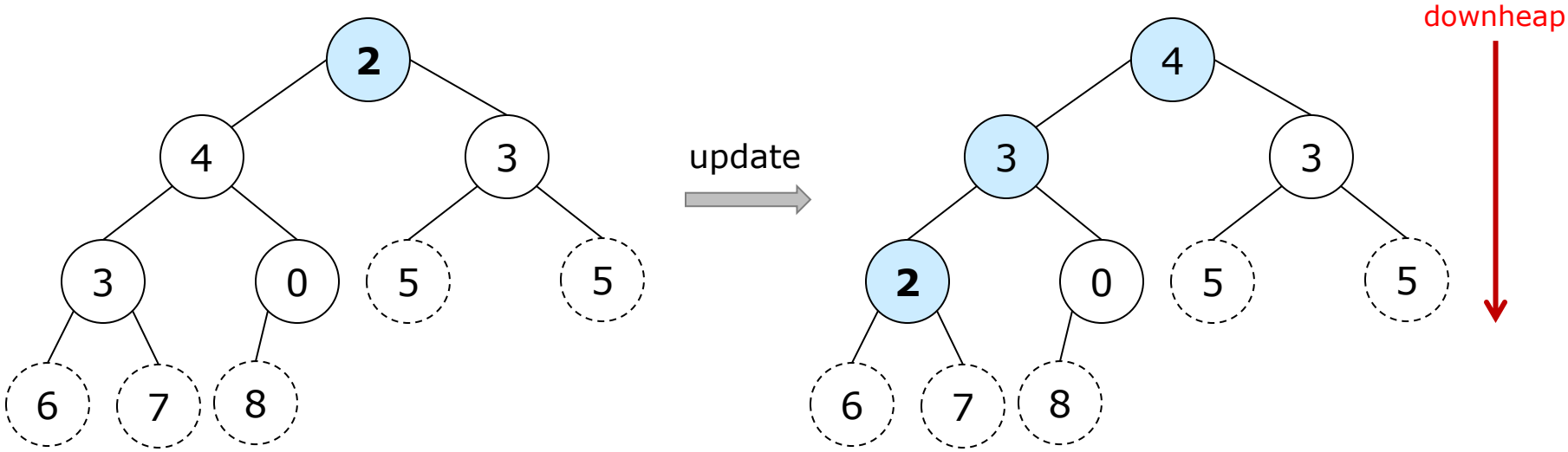
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



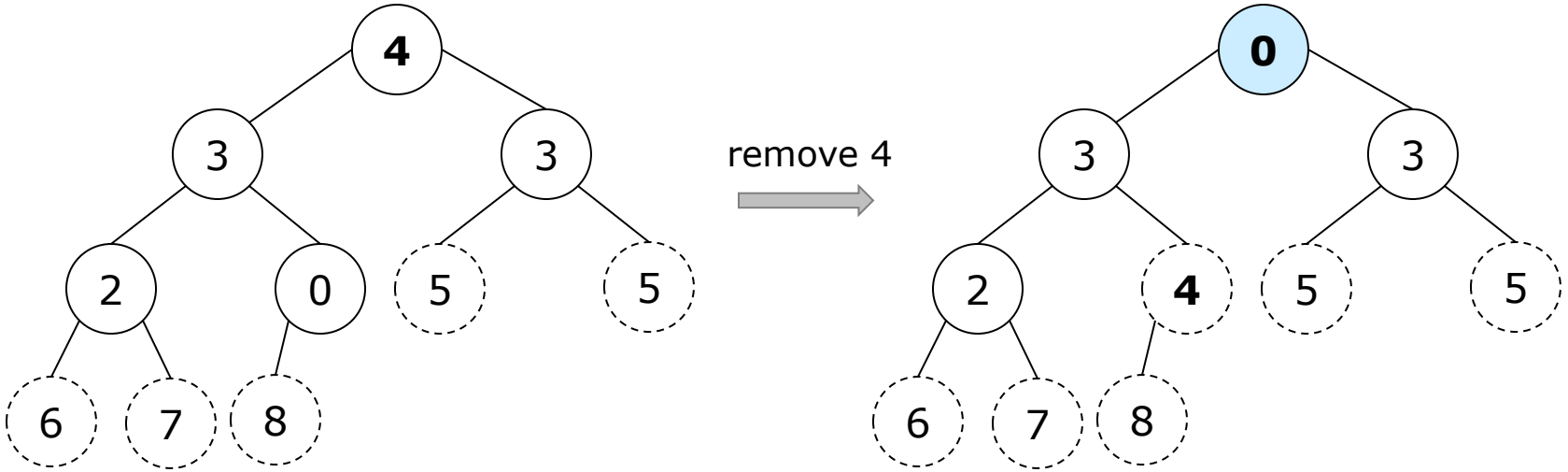
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



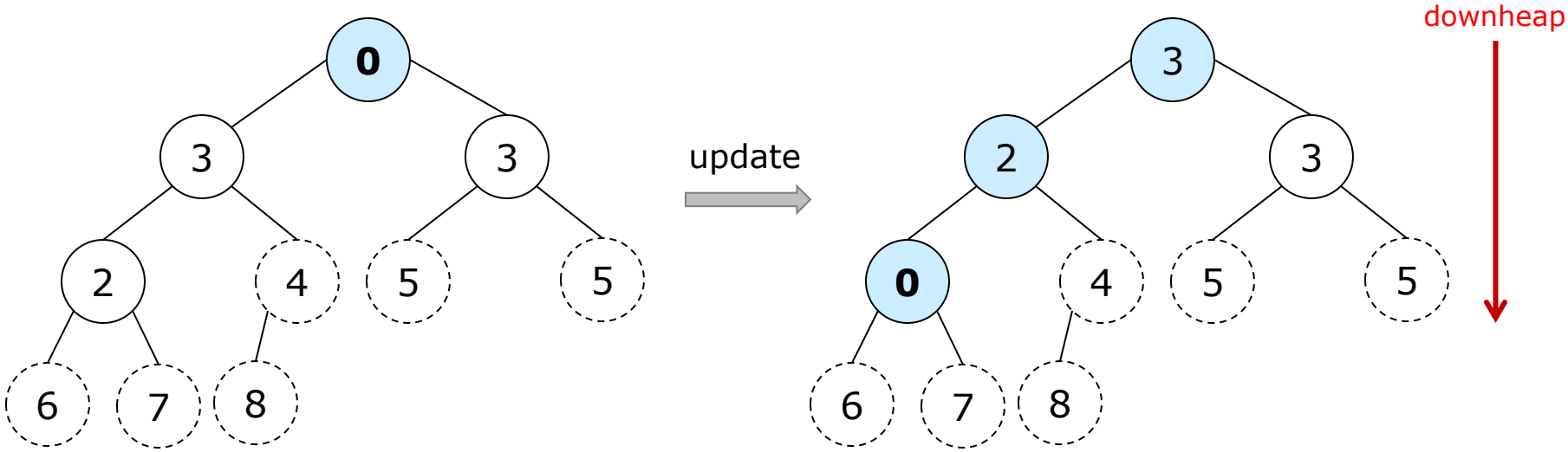
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



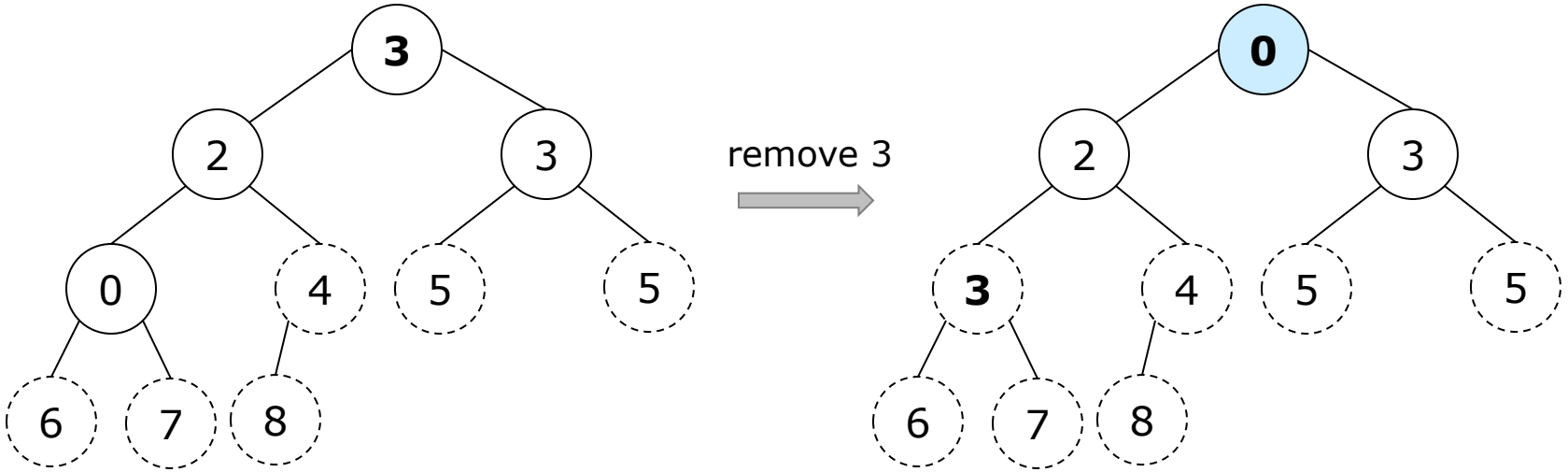
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



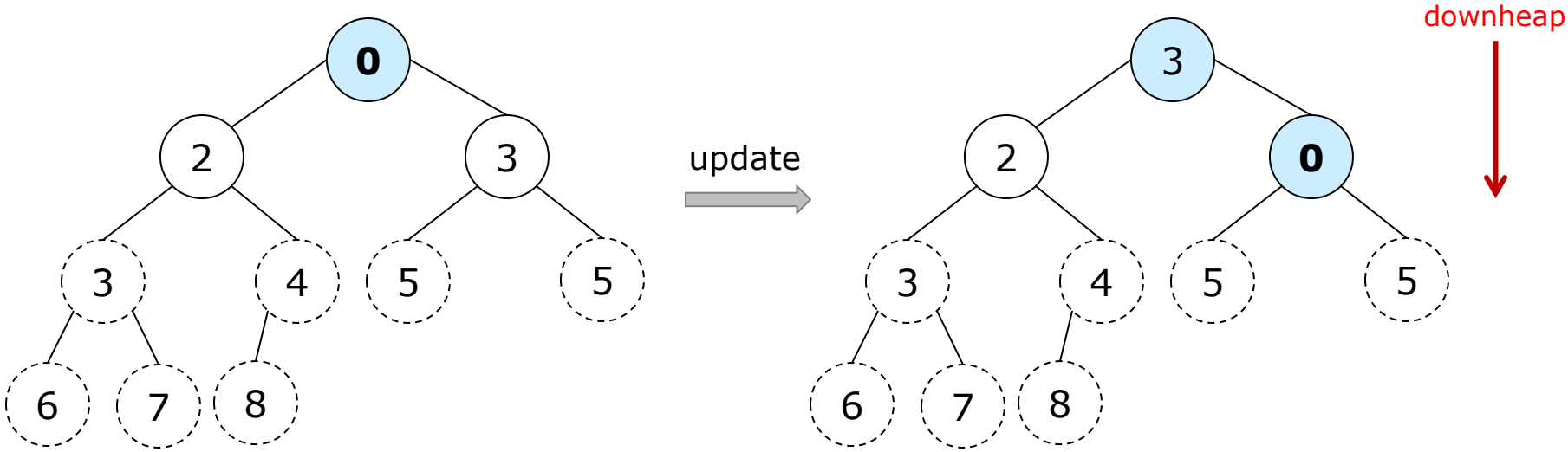
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



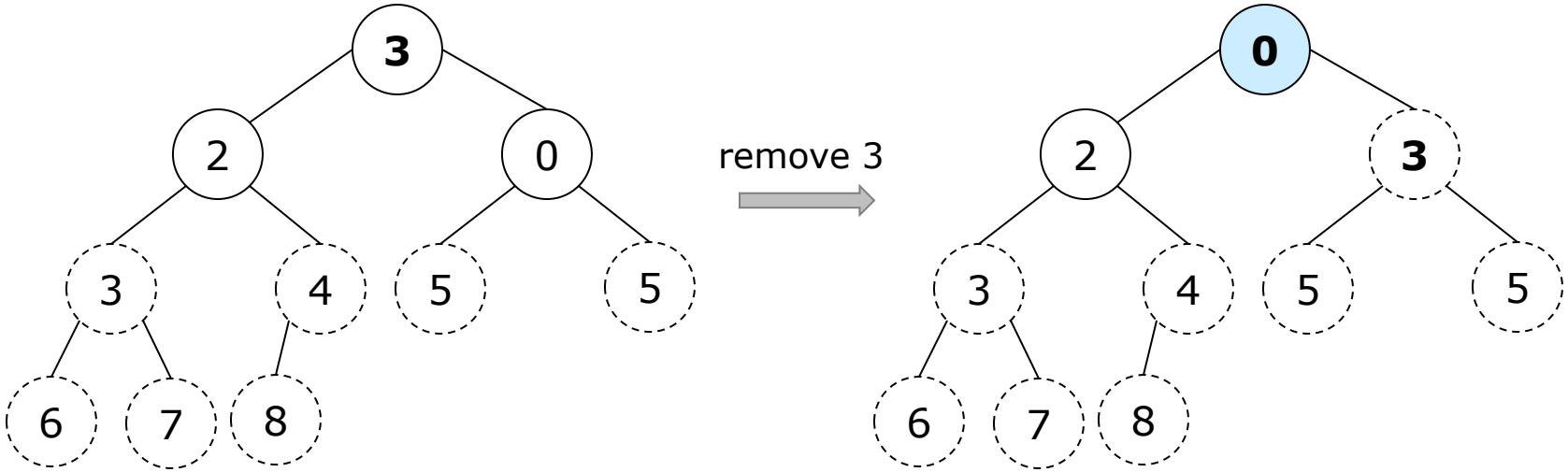
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



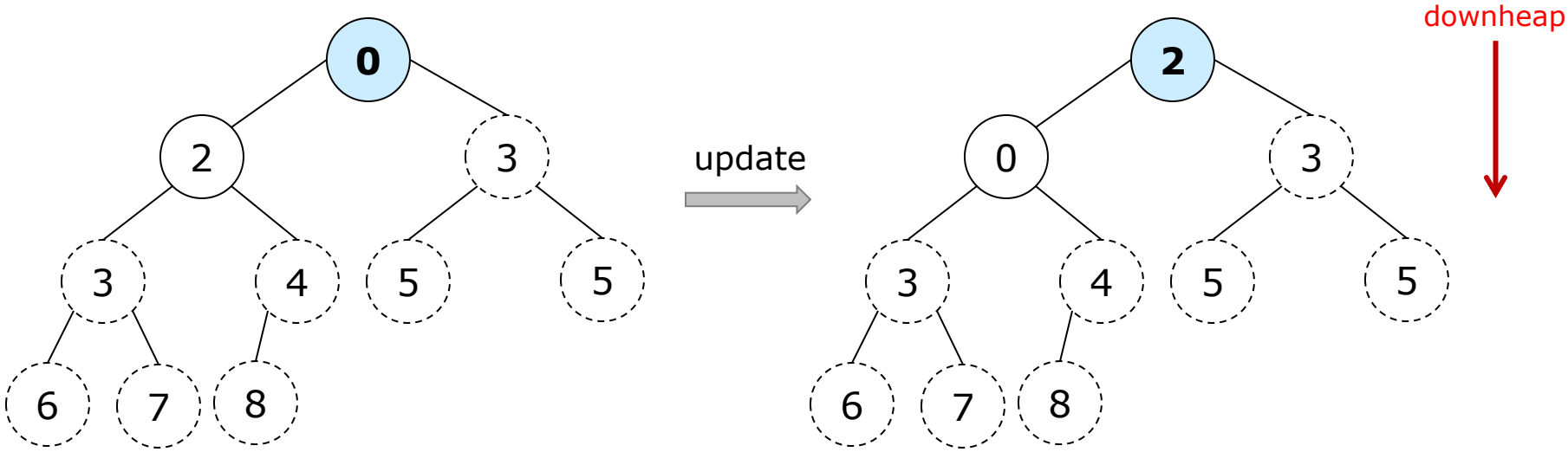
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



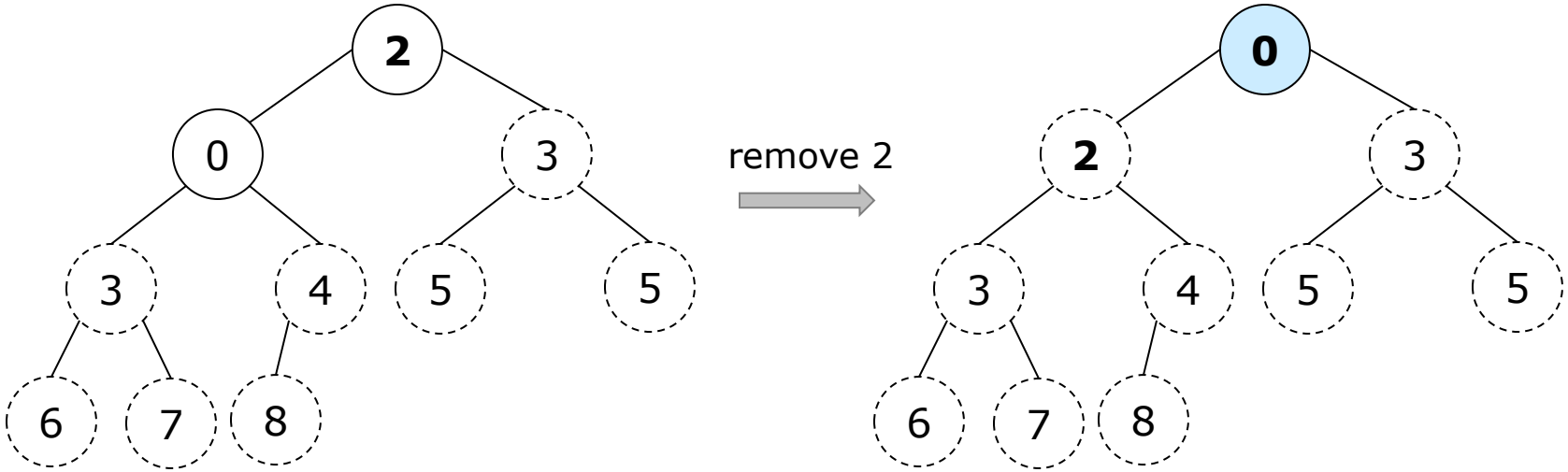
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



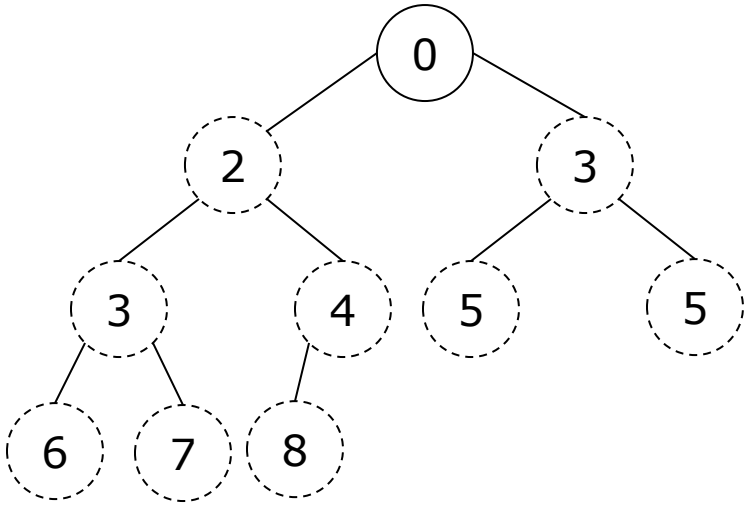
4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



4.3. Heaps and heapsort

- Heapsort: 4, 3, 5, 2, 6, 5, 7, 0, 8, 3



Sorted array: 0, 2, 3, 3, 4, 5, 5, 6, 7, 8

4.3. Heaps and heapsort

- Time complexity to sort *n* elements with a heap
  - Abstract operation: *comparison*
  - Using a heap built with top-down heap construction

$$C_n = O(n\log_2 n) + 2n\log_2 n = O(n\log_2 n)$$

  - Using a heap built with bottom-up heap construction

$$C_n = O(n) + 2n\log_2 n = O(n\log_2 n)$$

4.3. Heaps and heapsort

- Your turn:

4.3.1. Trace by hand heapsort

(4.3.1.1) 23, 7, 92, 6, 12, 24, 40, 44, 20, 21

(4.3.1.2) 44, 30, 50, 22, 60, 55, 77, 55

(4.3.1.3) 2, 4, 6, 8, 10, 10

(4.3.1.4) 13, 11, 7, 7, 3, 2, 1

4.3. Heaps and heapsort

- Your turn:

  4.3.2. Is it always true that the bottom-up and top-down heap-construction algorithms yield the same heap for the same input? Check the previous results.

  4.3.3. Outline an algorithm for checking whether an array H[1..*n*] is a heap and determine its time efficiency.

  4.3.4. Design an efficient algorithm for finding and deleting an element of the smallest value in a max heap and determine its time efficiency.

  4.3.5. Design an efficient algorithm for finding and deleting an element of a given value *v* in a given heap H and determine its time efficiency.

4.4. Horner's rule for polynomial evaluation

Consider the problem of computing the value of a polynomial of degree *n*

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0 \qquad (4.1)$$

at a given point x.

G.H. Horner, the British mathematician, 150 years ago, proposed an efficient method for evaluating a polynomial of degree *n*.

From (4.1), we can obtain a new formula by successively taking x as a common factor in the remaining polynomials of diminishing degrees.

$$p(x) = (\ldots(a_n x + a_{n-1})x + \ldots)x + a_0 \qquad (4.2)$$

Horner's rule: *representation change*

a polynomial of degree *n* ➔ a polynomial of degree *1*

4.4. Horner's rule for polynomial evaluation

ALGORITHM *Horner*$(P[0..n], x)$

//Evaluates a polynomial at a given point by Horner's rule
//Input: An array $P[0..n]$ of coefficients of a polynomial of degree n
// (stored from the lowest to the highest) and a number x
//Output: The value of the polynomial at x
$p \leftarrow P[n]$
for $i \leftarrow n-1$ **downto** 0 **do**
  $p \leftarrow x * p + P[i]$
return p

4.4. Horner's rule for polynomial evaluation

$p1(x) = 2x^4 - x^3 + 3x^2 + x - 5$

$p1(x) = (2x^3 - x^2 + 3x + 1)x - 5$

$p1(x) = ((2x^2 - x + 3)x + 1)x - 5$

$p1(x) = (((2x - 1)x + 3)x + 1)x - 5$

$p1(x) = ?$ at $x = 3$

$A = 2x + (-1) = 5$
$B = Ax + 3 = 18$
$C = Bx + 1 = 55$
$D = Cx + (-5) = 160$

$p1(x) = D = 160$

4.4. Horner's rule for polynomial evaluation

$p2(x) = 3x^4 - x^3 + 2x + 5$

$p2(x) = (3x^3 - x^2 + 0x + 2)x + 5$

$p2(x) = ((3x^2 - x + 0)x + 2)x + 5$

$p2(x) = (((3x - 1)x + 0)x + 2)x + 5$

$p2(x) = ?$ at $x = -2$

$A = 3x + (-1) = -7$
$B = Ax + 0 = 14$
$C = Bx + 2 = -26$
$D = Cx + 5 = 57$

$p2(x) = D = 57$

4.4. Horner's rule for polynomial evaluation

ALGORITHM *Horner*$(P[0..n], x)$

//Evaluates a polynomial at a given point by Horner's rule
//Input: An array $P[0..n]$ of coefficients of a polynomial of degree n
// (stored from the lowest to the highest) and a number x
//Output: The value of the polynomial at x
$p \leftarrow P[n]$
for $i \leftarrow n-1$ **downto** 0 **do**
  $p \leftarrow x * p + P[i]$
return p

- Time complexity of Horner's rule for evaluating a polynomial of degree *n* at a given point
  - Abstract operation: *multiplication*
  - Worst, Average, Best cases: O(*n*)

4.4. Horner's rule for polynomial evaluation

- Your turn:

  4.4.1. Evaluate a polynomial at a given point by Horner's rule

  - 4.4.1.1. P1 = $2x^4 - x^3 + 4x^2 + x - 5$ at x=3
  - 4.4.1.2. P2 = $3x^5 + x^4 - 5x^2 + 2x + 1$ at x=5
  - 4.4.1.3. P3 = $x^6 + 2x^5 - 3x^3 + x + 4$ at x=2
  - 4.4.1.4. P4 = $x^4 + 2x^2 + 5x - 1$ at x=4
  - 4.4.1.5. P5 = $-2x^5 + 3x^4 - x^3 + x^2 + 5x$ at x=3
  - 4.4.1.6. P6 = $3x^4 - x^3 + 2x + 5$ at x=-2

4.4. Horner's rule for polynomial evaluation

- Your turn:

4.4.2. Find the total number of multiplications and the total number of additions made by this algorithm.

ALGORITHM *BruteForcePolynomialEvaluation*$(P[0..n], x)$

```
//The algorithm computes the value of polynomial P at a given point x
//by the "highest to lowest term" brute-force algorithm
//Input: An array P[0..n] of the coefficients of a polynomial of degree n,
//       stored from the lowest to the highest and a number x
//Output: The value of the polynomial at the point x
p ← 0.0
for i ← n downto 0 do
    power ← 1
    for j ← 1 to i do
        power ← power * x
    p ← p + P[i] * power
return p
```

4.5. String matching by Rabin-Karp algorithm

- String matching: finding all occurrences of a pattern P in a text T
- A shift s shows that P occurs in T from $s+1$.
  - $T[s+1..s+m] = P[1..m]$ with $|T| = n$, $|P| = m$, and $0 \leq s \leq n - m$
- T = 2 3 5 9 0 2 3 1 4 1 5 2 6 3 5 9 0 2 1
- P = 3 5 9 0

➔ s = ???

4.5. String matching by Rabin-Karp algorithm

- String matching: finding all occurrences of a pattern P in a text T
- A shift s shows that P occurs in T from $s+1$.
  - $T[s+1..s+m] = P[1..m]$ with $|T| = n$, $|P| = m$, and $0 \le s \le n - m$
- T = 2 3 5 9 0 2 3 1 4 1 5 2 6 3 5 9 0 2 1
- P = 3 5 9 0
- ➔ s = *1, 13*

Rabin-Karp: elementary number-theoretic notions such as the equivalence of two numbers after we "modulo" the third number

4.5. String matching by Rabin-Karp algorithm

- Rabin-Karp algorithm
  - Assume: $\Sigma = \{0, 1, 2, \ldots, 9\}$ and the elements of P and T are characters drawn from a finite alphabet Σ.
    - In general, a character is a digit in radix-d notation where $d = |\Sigma|$.
    - We can view a string of k consecutive characters as a length-k decimal number.
      - Character string "3590" corresponds to decimal number 3,590.
  - Given a pattern $P[1..m]$, let p denote its corresponding decimal value.
  - Given a text $T[1..n]$, let t_s denote the decimal value of the length-m substring $T[s+1...s+m]$, for $s = 0, 1, .., n-m$.
  - $t_s = p$ if and only if $T[s+1..s+m] = P[1..m]$.
  - s is a valid shift if and only if $t_s = p$.

4.5. String matching by Rabin-Karp algorithm

- For $P[1..m]$, p is computed in $O(m)$ with Horner's rule (P: a polynomial of degree m-1 and $x=d=10$):

$$p = ((..(P[1]*10 + P[2])*10 +..+ P[m-2])*10 + P[m-1])*10 + P[m]$$

- For $T[1...m]$, $t_s = t_0$ is similarly computed in $O(m)$.
- For $T[s+1..m+s]$, t_{s+1} is then computed from t_s with $s=1..n-m$:

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1]$$

T = 2 3 5 9 0 2 3 1 4 1 5 2 6 3 5 9 0 2 1

P = 3 5 9 0

For $T[2..5]$, $t_s = t_1 = 3590$

For $T[3..6]$, $t_{s+1} = t_2 = 10(t_1 - 10^{4-1}T[2]) + T[1+4+1]$

$= 10(3590-1000*3) + 2 = 5902$

$s = 1 \rightarrow$ T = 2 3 5 9 0 2 3 1 4 1 5 2 6 3 5 9 0 2 1

$s = 2 \rightarrow$ T = 2 3 5 9 0 2 3 1 4 1 5 2 6 3 5 9 0 2 1

4.5. String matching by Rabin-Karp algorithm

- In practice, p and t_s may be too large. Fortunately, there is a simple cure for this problem. We compute p and the t_s values *modulo* a suitable modulus q.
- The modulus q is chosen as a *prime* such that $10q$ just fits within a computer word for single-precision arithmetic.
- In general, with a d-ary alphabet $\{0, 1, \ldots, d-1\}$, we choose q such that dq fits within a computer word.

4.5. String matching by Rabin-Karp algorithm

- t_{s+1} is now computed from t_s using "modulo":

  $t_{s+1} = (d(t_s - h\mathrm{T}[s+1]) + \mathrm{T}[s+m+1]) \bmod q$

  where $h = d^{m-1} \bmod q$
- $t_s \equiv p \bmod q$ does *not* imply that $t_s = p$.
- If $t_s \neq p \bmod q$ then definitely $t_s \neq p$.
  - Such a shift s is *invalid*.

➔ Check $t_s \equiv p \bmod q$ to rule out *invalid* shifts s

➔ Any shift s for which $t_s \equiv p \bmod q$ must be checked *further* to see if s is really a *valid hit* or just a *spurious hit*.

4.5. String matching by Rabin-Karp algorithm

```
procedure RABIN-KARP-MATCHER(T, P, d, q);
/* T is the text, P is the pattern, d is the radix and q is the prime */
begin
    n: = |T|; m: = |P|;
    h: = d^(m-1) mod q;
    p: = 0; t_0 : = 0;
    for i: = 1 to m do
    begin
        p: = (d*p + P[i])mod q;
        t_0: = (d*t_0 + T[i])mod q
    end
    for s: = 0 to n – m do
    begin
        if p = t_s then  /* there may be a hit  */
              if P[1..m] = T[s+1..s+m] then
                    Print "Pattern occurs with shift "s;
        if s < n – m then
          t_(s+1): = (d(t_s –T[s + 1]h) + T[s+m+1])mod q
    end
end
```

4.5. String matching by Rabin-Karp algorithm

```
procedure RABIN-KARP-MATCHER(T, P, d, q);
/* T is the text, P is the pattern, d is the radix and q is the prime */
begin
    n: = |T|; m: = |P|;
    h: = d^(m-1) mod q;
    p: = 0; t_0 : = 0;
    for i: = 1 to m do
    begin
        p: = (d*p + P[i])mod q;
        t_0: = (d*t_0 + T[i])mod q
    end
    for s: = 0 to n – m do
    begin
        if p = t_s then  /* there may be a hit  */
              if P[1..m] = T[s+1..s+m] then
                    Print "Pattern occurs with shift "s;
        if s < n – m then
          t_{s+1}: = (d(t_s –T[s + 1]h) + T[s+m+1])mod q
    end
end
```

(a mod n) mod n = a mod n

(a+b) mod n = ((a mod n) + (b mod n)) mod n

(ab) mod n = ((a mod n) (b mod n)) mod n

h = d^{m-1} mod q = (d mod q) ((d^m-2) mod q)

h = ((d mod q) (d mod q) .. (d mod q)) mod q

4.5. String matching by Rabin-Karp algorithm

T = 2 3 5 9 0 2 3 1 4 1 5 2 6 3 5 9 0 2 1

P = 3 5 9 0 , q = 13, d = 10, $h = d^{m-1} \bmod q = 1000 \bmod 13 = 9$

t_{s+1}: = (d(t_s –T[s + 1]h) + T[s+m+1])mod q

Shift	Pattern	T[s+1]	T[s+m+1]	t_s (=? p mod q = 2)	Hit?	True hit?
0	2359	-	-	6	FALSE	
1	**3590**	**2**	**0**	**2**	**TRUE**	**yes**
2	5902	3	2	0	FALSE	
3	9023	5	3	1	FALSE	
4	0231	9	1	10	FALSE	
5	2314	0	4	0	FALSE	
6	3141	2	1	8	FALSE	
7	1415	3	5	11	FALSE	
8	4152	1	2	5	FALSE	
9	1526	4	6	5	FALSE	
10	5263	1	3	11	FALSE	
11	2635	5	5	9	FALSE	
12	*6359*	*2*	*9*	*2*	*TRUE*	*no*
13	**3590**	**6**	**0**	**2**	**TRUE**	**yes**
14	5902	3	2	0	FALSE	
15	9021	5	1	12	FALSE	

4.5. String matching by Rabin-Karp algorithm

- Time complexity of Rabin-Karp for finding all the occurrences of a pattern P in a string T where |T| = n and |P| = m
  - Abstract operation: *comparison*
  - Worst case when every shift is verified.

$$O((n-m+1)m)$$

  - Average to Best case when few (c shifts) are verified.

$$O((n-m+1) + cm) = O(n+m)$$

4.5. String matching by Rabin-Karp algorithm

- Your turn:

  4.5.1. Use Rabin-Karp algorithm to determine the occurences of P in T in the decimal system. How many spurious matches are there? Given q = 11. How about q = 13?

  T = 3141592653589793

  P = 26

4.5. String matching by Rabin-Karp algorithm

- Your turn:

  4.5.2. Use Rabin-Karp algorithm to determine the occurences of P in T in the hexadecimal system. How many spurious matches are there? Given q = 17.

  T = 31ABCEF926CEF9D897A

  P = CEF9

Chapter 4. Transform-and-Conquer

- Your turn:

  4.5.3. Use Rabin-Karp algorithm to determine the occurences of P in T in the hexadecimal system. How many spurious matches are there? Given q = 23.

- T = A31AB7EF926ACEF9264
- P = EF926. What if P = 1AB7E?

Chapter 4. Transform-and-Conquer
Problem Reduction

- Your turn:
- (i) Consider the problem of *finding*, for a given positive integer *n*, *the pair of integers* whose sum is *n* and whose product is as large as possible. Design an efficient algorithm for this problem and indicate its efficiency class.
- (ii) The graph-coloring problem is usually stated as the vertex-coloring problem: assign the smallest number of colors to vertices of a given graph so that no two adjacent vertices are the same color. Consider the edge-coloring problem: assign the smallest number of colors possible to edges of a given graph so that no two edges with the same endpoint are the same color. Explain how the edge-coloring problem can be reduced to a vertex-coloring problem.

Algorithms Analysis and Design (CO3031)

- Chapter 4. Transform-and-conquer
  - 4.1. Transform-and-conquer strategy
  - 4.2. Gaussian Elimination for solving a system of linear equations
  - 4.3. Heaps and heapsort
  - 4.4. Horner's rule for polynomial evaluation
  - 4.5. String matching by Rabin-Karp algorithm